

Würzburger Institut für Verkehrswissenschaften
(Institute for Traffic Sciences)



Robert-Bosch-Straße 4, 97209 Veitshöchheim
Kontakt: silab@wiw.de

SILAB[®] **Referenz Visuali- sierung SGE**

Version 7.1

Stand: September 2022

INHALT

1	ÜBERBLICK.....	4
1.1	Grundsätzlicher Aufbau	4
1.2	Die SGE 3D-Engine	4
1.2.1	Bibliotheken	4
1.2.2	DPU's	4
1.3	Die Visualisierungen	5
1.3.1	Bibliotheken	5
1.3.2	DPU's	5
2	KONFIGURATION.....	7
2.1	Einbindung in die SILAB-Basiskonfiguration	7
2.1.1	Grundlagen	7
2.1.2	Beispiel	7
2.2	Hardwaresupport und Konfiguration der Grafikkarten	9
2.2.1	Grafik-Konfiguration	10
2.2.2	Grafikeinstellungen	12
2.2.3	NVidia, Aktuelle Treiberversionen	12
2.2.4	ATI, Catalyst-Treiberversionen	13
2.3	Threads	15
2.4	Kamera	15
2.5	Verzeichnisstruktur	16
2.6	Textur-Cache	18
2.7	Taktung der Rechner	18
2.8	Geometrieverzerrung	19
2.9	Weitere Konfiguration der Visualisierung	20
3	ERWEITERUNG UM GRAFISCHE OBJEKTE	21
3.1	Objekttypen	21
3.2	Grundlagen zum Objektentwurf	22
3.2.1	Einteilung in Unterobjekte	22
3.2.2	Vertextabelle und Primitive	22
3.2.3	Koordinatensysteme	22
3.2.4	Texturen und Mipmaps	23
3.3	Formale Objektdefinition	27
3.3.1	Syntax der Objektdateien	27
3.3.2	Objektdefinition	27

3.3.3	Parameter	28
3.3.4	Cookies	42
3.3.5	Shader-Parameter	42
3.3.6	Benutzerdefinierte Parameter	43
3.3.7	Vertextabelle	43
3.3.8	Primitive	46
3.3.9	Subobjekte	49
3.4	Beispielobjekt	50
3.4.1	Einfaches Objekt: Haus	50
4	SGEOBJECTPREV	53
4.1	Überblick	53
4.2	Laden eines Objekts	54
4.2.1	„Nach dem Laden des Objekts sehe ich nichts!“	54
4.3	Warn- und Fehlermeldungen	54
4.4	Navigation	55
4.5	Import von 3D-Modellen	56
4.5.1	„Nach dem Import des Objekts sehe ich nichts!“	58
4.5.2	Spezielle Funktionen in 3D-Modellen	58
4.6	Tools zur Nachbearbeitung	59
4.6.1	Objekt transformieren...	59
4.6.2	Objekt XY-zentrieren	59
4.6.3	Subobjekte verschmelzen...	59
4.6.4	CFG kopieren	60
4.6.5	Batch-Import für AEdit	60
5	GENERIERUNG VON SCREENSHOTS	61
6	GENERIERUNG DIGITALER VIDEOS.....	62

1 ÜBERBLICK

Als Visualisierung wird der Programmteil von SILAB bezeichnet, der dafür sorgt, dass auf den Anzeigegeräten des Fahrsimulators (Monitore oder Projektoren) eine realistische Darstellung des simulierten Szenarios erscheint.

1.1 Grundsätzlicher Aufbau

Die SILAB-Visualisierung besteht aus mehreren Programm-Modulen, die auf den Grafikrechnern des Simulators ausgeführt werden.

Die Visualisierung lässt sich zunächst grob in zwei Teile unterteilen:

- Die 3D-Engine SGE bietet grundlegende Funktionen, um mehrere tausend grafische 3D-Objekte in eine „Simulationswelt (*SGEWorld*)“ zu laden und diese in Echtzeit auf dem Bildschirm darzustellen und zu animieren (*SGEView*). Sie baut dazu auf den herstellerübergreifenden Grafikstandard OpenGL auf, der von allen modernen 3D-Grafikkarten unterstützt wird, um eine hardwarebeschleunigte Darstellung zu erreichen.
- Die Simulationswelt wird von den Visualisierungen *SCNXSGE*, *TRFSGE*, *MOPSGE*, etc. der entsprechenden SILAB-Module *SCNX*, *TRFX* und *MOV* mit 3D-Objekten gefüllt.

1.2 Die SGE 3D-Engine

Die 3D-Grafik-Engine SGE besteht aus mehreren Bibliotheken und DPUs, welche unterschiedliche Aufgaben übernehmen.

1.2.1 Bibliotheken

SGEDriver

Hardwarenahe Funktionen zur Ansteuerung der Grafikhardware.

SGEAdministration

Grundlegende Verwaltungsstrukturen für Grafikobjekte, Animationen und Texturen.

SGEEngine

Verwaltung der *SGEWorld* und der *SGEViews*.

Die SGE-Bibliothek verwaltet eine globale Welt, in der die Objekte eingefügt und bewegt werden. Die Visualisierungen fügen ihre geladenen Objekte in die *SGEWorld* ein und positionieren sie dort im Weltkoordinatensystem der Fahrsimulation.

1.2.2 DPUs

DPUSGEWorld

Diese DPU erzeugt die globale *SGEWorld* und sorgt für die Triggerung der Welt während der Simulation.

DPUSGEView

Auf jedem Grafikrechner können maximal sechs verschiedene Ansichten der *SGEWorld* angezeigt werden¹, die jeweils in einem eigenen Fenster und übli-

¹ Für Arbeitsplatz-Lizenzen ist die Anzahl von *DPUSGEView*-Instanzen auf zwei beschränkt.

cherweise auf einem eigenen Bildschirm dargestellt werden. Für jede Ansicht wird eine Instanz der DPUSGEView benötigt. Sie positioniert eine virtuelle Kamera in der SGWorld. Das durch die Kamera gesehene Abbild der Welt wird im Fenster des Views dargestellt.

Zuerst werden alle 3D-Objekte gerendert. Das hieraus resultierende Bild wird anschließend mit den 2D-Overlayobjekten überlagert und im Fenster dargestellt.

Jeder View arbeitet mit zwei Bildern (*Double-Buffering*): während das zuletzt erzeugte Bild angezeigt wird, wird im Hintergrund bereits die nächste Ansicht auf die Welt vorbereitet.

DPUSGEDistortionXXX

Diese DPUs ermöglichen eine geometrische Verzerrung des erzeugten Bilds, wodurch eine Projektion auf unterschiedliche Leinwandformen (zylindrisch, sphärisch, ...) möglich ist. Näheres zur Geometrieverzerrung finden Sie in Kap. 2.8.

DPUSGEProjection

Rechnet 3D-Weltkoordinaten in 2D-Overlaykoordinaten um. (Eine genauere Erklärung der Koordinatensysteme in SGE finden Sie in Kap. 3.2.3).

1.3 Die Visualisierungen

Für die Visualisierung der SILAB-Module SCNX, TRFX und MOV gibt es die entsprechenden Visualisierungsmodule SCNXSGE, TRFSGE und MOPSGE.

Auch diese sind wiederum in Bibliotheken und zugehörige DPUs aufgeteilt.

1.3.1 Bibliotheken

SCNXSGE

Die Visualisierung SCNXSGE stellt die Datenbasis SCNX von SILAB grafisch dar. Dies umfasst die Darstellung der umgebenden Landschaft (Erdoberfläche), der Straßen und Verkehrsknoten (Course- und Area2-Instanzen), der Landschaftsobjekte (Bäume, Häuser etc.) und des Hintergrund-Panoramas inklusive Himmel und Wolken.

SCNXSGE bildet den komplexesten Teil der Visualisierung und kann über die „SCNXSGE Params“ und die „SCNX Params“ ausführlich konfiguriert werden (siehe „Referenz Datenbasis SCNX“).

TRFSGE

Die Bibliothek TRFSGE stellt Fahrzeuge des umgebenden Verkehrs grafisch dar, die von der Verkehrssimulation TRFX von SILAB gesteuert werden. Auch die Lichter der Fahrzeuge und Ampeln werden von ihr verwaltet.

MOPSGE

Die grafische Darstellung der Fußgänger, die durch das SILAB-Modul MOV simuliert werden, wird von MOPSGE durchgeführt.

1.3.2 DPUs

Zu den Visualisierungen SCNXSGE, TRFSGE und MOPSGE existiert jeweils die DPU **DPUSCNXSGE**, **DPUTRFSGE** und **DPUMOPSGE**, die für die ordnungsgemä-

ße Einbindung der Visualisierungen in die Simulation sorgen. Die Visualisierung von Wetterphänomenen wie Regen und Blitz übernimmt die **DPUSGEWeather**.

Weitere DPUs sind dazu geeignet, grafische Objekte, die vom SILAB-Anwender erstellt wurden, in die Simulation einzubinden:

DPUSGEObject

Mit der **DPUSGEObject** können Sie einzelne grafische Objekte einfach in die SGEWorld einbinden, ohne erst eine eigene DPU entwickeln zu müssen. Über DPU-Eingänge wird außerdem eine komfortable Möglichkeit bereitgestellt, das Objekt frei zu positionieren, zu drehen und zu skalieren. Da die Eingangsvariablen während der Simulation verändert werden können, bietet sie auch eine Möglichkeit zur Entwicklung einfacher Animationen.

DPUSCNXObjectsSGE

Ähnlich zur **DPUSGEObject** können hiermit einzelne grafische Objekte in die SGEWorld eingebunden werden. Allerdings wird die Positionierung bei der **DPUSCNXObjectsSGE** relativ zur Fahrstrecke vorgenommen, d.h. eine Positionierung ist an einem bestimmten Streckenmeter und Abstand von der Straßenmitte möglich.

Eine weitere Gruppe von DPUs ist besonders zur Einblendung von „Head-Up-Displays“ (HUD), d.h. zweidimensionalen Grafiken, die als Overlay-Objekte über die Szenerie gezeichnet werden, geeignet.

DPUSGESTring

Ermöglicht eine Textausgabe. Ein frei definierbarer, möglicherweise mehrzeiliger Text kann in wählbarer Schriftart und Farbe dargestellt werden. In den Text kann der aktuelle Wert einer Variablen als Zahl eingebunden werden, z.B. um eine digitale Tachoanzeige („47 km/h“) zu realisieren.

DPUSGEInstrumentPanel

Ermöglicht die Einblendung einer Instrumentenanzeige in die grafische Ansicht (Armaturenbrett mit Lämpchen und Symbolanzeigen).

DPUSGERoundGauge

Ermöglicht die Einblendung eines Rundinstruments (z.B. Tachometer) in die grafische Ansicht.

2 KONFIGURATION

2.1 Einbindung in die SILAB-Basiskonfiguration

2.1.1 Grundlagen

Als Teil der Standard-Konfiguration von SILAB, die wir auf Ihrem Simulator eingerichtet haben, wurde das grafische Subsystem vorkonfiguriert. Diese Konfiguration funktioniert wie folgt:

1. Auf allen Grafikrechnern wird zunächst die DPUSGEWorld gestartet, um die SGEWorld bereitzustellen.
2. Außerdem werden alle gewünschten Visualisierungen instanziiert (DPUSCNXSGE, DPUTRFSGE, DPUMOPSGE und ggf. DPUSGEObject), um Objekte in die SGEWorld einzufügen und zu animieren.
3. Ansichten auf die SGEWorld werden durch die Einbindung von DPUSGE-View oder DPUSGERecorder erzeugt.

Während jedem Simulationsschritt müssen die DPUs in dieser Reihenfolge aufgerufen werden, damit stets schnellstmöglich der aktuelle Zustand von Datenbasis und Verkehr dargestellt wird. Um dies zu erreichen, werden die Index-Einträge der DPUs aufsteigend gesetzt.

2.1.2 Beispiel

Als Beispiel wird eine Konfiguration der Visualisierung für einen Fahrsimulator mit drei Projektoren und jeweils einem zugehörigen Grafikrechner vorgestellt. Der Hardwareaufbau ist in Abbildung 1 dargestellt.

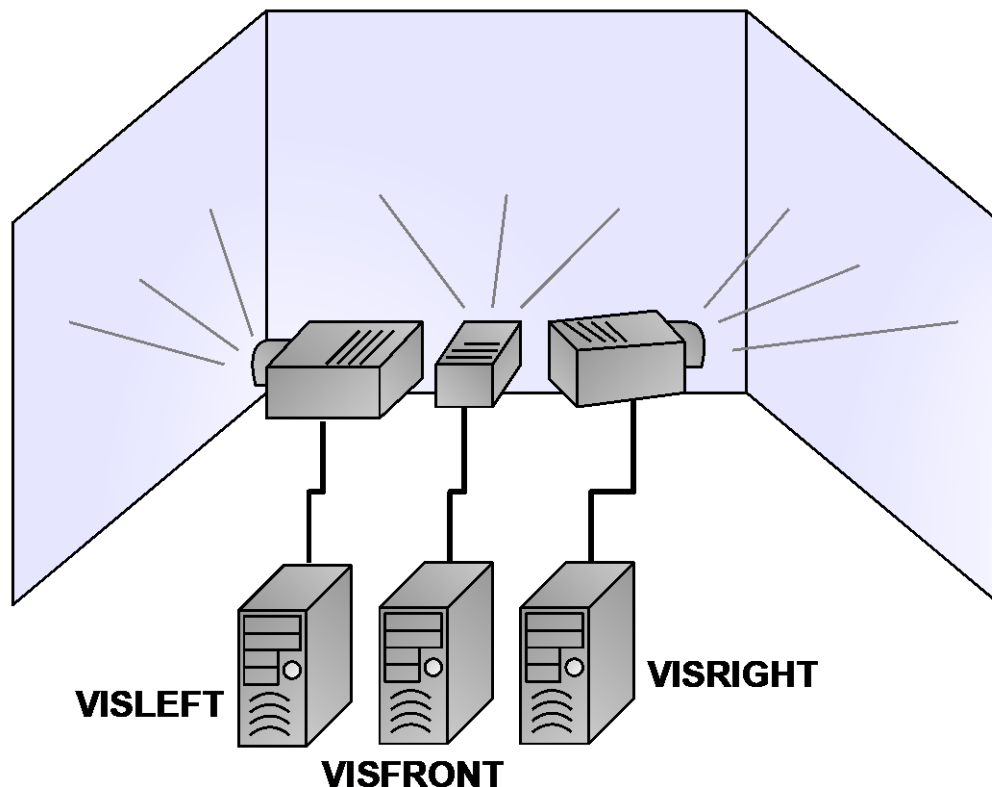


Abbildung 1 Hardwareaufbau eines einfachen 3-Kanal Simulators

In der bereits vorinstallierten SILAB-Standard-Konfiguration werden die Visualisierungs-DPUs wie folgt eingebunden:

- Die DPUSGEWorld (3D-Umgebung) und die DPUSGEView (Ansicht) werden in der Datei `lib\configurations\system\DPUSGE.inc` eingebunden.
- Die darauf aufbauenden Visualisierungen DPUSCNXSGE, DPUTRFSGE und DPUMOPSGE werden in den Dateien `DPUSCNX_SGE.inc`, `DPUTRFX_SGE.inc` und `DPUMOV_SGE.inc` im selben Verzeichnis eingebunden.

Hinweis: Sie sollten an diesen Dateien keine Änderungen vornehmen, da diese bei einem Update auf eine neue SILAB-Version überschrieben werden.

Zur Konfiguration der Visualisierung für diesen Simulator werden zwei Dateien angepasst:

Anpassungen der Computerconfiguration (COMPBase.inc)

In der Datei `lib\configurations\system\COMPBase.inc` werden die Grafikrechner eingebunden. Jedem Grafikrechner werden dabei mindestens die Aliasnamen `SCN` und `VIS` zugeordnet, damit die Datenbasis verfügbar ist und die Visualisierungs-DPUs auf dem Rechner ausgeführt werden.

Beispiel:

```
Pool Simulator
{
    Executable = true;
    # Grafikrechner Front-Links
    Computer VISLEFT
    {
        IP = "192.168.2.21";
        Frequency = 60;
        Alias = {ALL,SCN,VIS,VISF02};
    };
    # Grafikrechner Front-Rechts
    Computer VISRIGHT
    {
        IP = "192.168.2.22";
        Frequency = 60;
        Alias = {ALL,SCN,VIS,VISF03};
    };
    # Grafikrechner Front-Mitte
    Computer VISFRONT
    {
        IP = "192.168.2.23";
        Frequency = 60;
        Alias = {ALL,SCN,VIS,VISF01};
    };
    [...]
};
```

Hinweis:

Zusätzlich werden in diesem Beispiel die Aliasnamen `VISFnn` vergeben. Diese nummerieren alle Visualisierungen der Frontsicht (d.h. ohne Spiegel) durch, wobei `nn=01` immer die mittlere Ansicht ist. Dies ist nützlich, wenn bestimmte Einblendun-

gen (z.B. Head-Up-Displays mit Hilfe der DPUHUDSGE) nur auf einer der Ansichten zu sehen sein sollen.

Systemspezifische DPU-Parameter (SYSSGEPARAMS.INC)

Systemspezifische Parameter für die DPUs der Visualisierung werden in die Datei `lib\configurations\system\SYSSGEPARAMS.INC` eingetragen.

Typischerweise werden hier folgende Werte angepasst:

- Auflösung und Perspektive der Ansichten
- Vorgabe von Standardwerten für Optionen, die die Grafikleistung beeinflussen (z.B. Auswahl eines Rendermodus und Konfiguration von Antialiasing). Bei Bedarf können diese Optionen dann später in der DPU-Konfiguration der einzelnen Projekte überschrieben werden.

Beispiel:

```
# Festlegen der Größe aller Ansichten (Angaben in Pixeln)
SGEView.WindowW = 1400;
SGEView.WindowH = 1050;

# Festlegen des Blickwinkels der einzelnen Ansichten
# entsprechend des Hardware-Aufbaus
VISFRONT.SGEView.CamGamma = 0;
VISLEFT.SGEView.CamGamma = -60;
VISRIGHT.SGEView.CamGamma = 60;

# Vorgabe einiger leistungsabhängiger Parameter
# je nach Typ der verbauten Grafikkarten
SGEWorld.RenderMode = 4;
SGEWorld.AntiAlias = 8;
SGEWorld.ShadowQuality = 3;
```

Für eine ausführlichere Beschreibung der DPU-Parameter und –Eingänge sehen Sie bitte in den entsprechenden Dokumenten zu DPUSGEWorld, DPUSGEObject, DPUSGEView und DPUSGERecorder nach. Die Visualisierungen DPUSCNXSGE, DPUTRFSGE und DPUMOPSGE müssen (und können) nicht weiter über DPU-Parameter konfiguriert werden. Auch eine Verschaltung der Eingänge ist hier nicht nötig, da alle Daten durch interne Funktionsaufrufe direkt aus den entsprechenden Modulen *SCNXSGE*, *TRFClient* und *MOVClient* bezogen werden.

2.2 Hardwaresupport und Konfiguration der Grafikkarten

Die SGE 3D Grafik-Engine benötigt mindestens Unterstützung für OpenGL 3.3. Folgende Grafikkarten werden unterstützt:

Nvidia

GeForce Serie 900	Ab Treiberversion GeForce 375. Für komplexe Szenarien wird mindestens die GeForce GTX 970 benötigt.
GeForce Serie 1000	Ab Treiberversion GeForce 381. Für komplexe Szenarien wird mindestens die GeForce GTX 1060 benötigt.

GeForce Serie 2000/1600 Ab Treiberversion GeForce 460. Für komplexe Szenarien wird mindestens die GeForce RTX 2060 bzw. GTX 1660 benötigt.

GeForce Serie 3000 Ab Treiberversion GeForce 460. Für komplexe Szenarien wird mindestens die GeForce RTX 3060 benötigt.

Bitte beachten Sie, dass die aktuelle Version der SGE Grafik-Engine keine Raytracing-Funktionalität (GeForce RTX-Karten) nutzt. Es ergibt sich daher neben der teils deutlich höheren Anzahl von Shaderkernen kein grundsätzlicher Vorteil durch den Einsatz von RTX- anstelle von GTX-Karten.

AMD

Aktuelle Radeon-Grafikkarten liefern eine vergleichbare Performance und visuelle Darstellung wie die entsprechenden GeForce-Grafikkarten. Es wurden allerdings keine ausführlichen Tests mit Radeon-Grafikkarten vorgenommen, so dass wir diese für den Einsatz in Simulatoren nicht empfehlen können.

2.2.1 Grafik-Konfiguration

Für bestimmte, insbesondere seltene, Grafikkarten muss die Konfiguration der Grafikengine SGE angepasst werden.

Hierfür kann die Datei `SYSSGE.inc` im Verzeichnis

`(D:\SILAB\)\lib\configurations\system` verwendet und bei Bedarf angelegt werden.

Folgende Konfigurationsoptionen können in dieser Datei eingegeben werden:

<i>NoGenMipmap</i>	Verwendet die Funktion <i>glGenerateMipmap()</i> nicht.
<i>NoTextureLODControl</i>	Verwendet die Einstellung <i>GL_TEXTURE_MIN_LOD/_MAX_LOD</i> nicht.
<i>MaxTextureSize</i>	Legt fest, welche Texturgröße maximal für 2D-Texturen verwendet werden soll. Es muss sich um eine Zweierpotenz handeln.
<i>MaxCubeTextureSize</i>	Legt fest, welche Texturgröße maximal für Cube-Map-Texturen verwendet werden soll. Es muss sich um eine Zweierpotenz handeln.
<i>OverrideHardwareCheck</i>	Versucht, die Grafikengine zu starten, obwohl die benötigten Features vom Grafiktreiber nicht bereitgestellt werden. Achtung: Schwere Darstellungsfehler und Programmabstürze könnten auftreten.

Die Optionen werden jeweils durch Eingabe des Wertes **true** aktiviert.

Beispiele:

Bei einigen Grafikkarten müssen Workarounds für fehlerhafte Funktionen manuell aktiviert werden. Tragen Sie dazu folgende Zeilen in die Datei `SYSSGE.inc` im Verzeichnis `(D:\SILAB\)\lib\configurations\system` ein:

```
SILAB_SGE {
```

```
NoTextureLODControl=true;
};
```

Falls bei der Darstellung komprimierter Texturen Fehler auftreten, sollte zunächst ein Update des Grafiktreibers durchgeführt werden. Falls dadurch keine Abhilfe erzielt wird, kann folgende Option eingestellt werden, um die Texturkompression komplett zu deaktivieren:

```
SILAB SGE {
    NoTextureCompression=true;
};
```

Dadurch erhöht sich allerdings der Speicherbedarf mindestens auf das 4-fache, was zu Störungen in der Simulation führen kann.

Für Grafikkarten mit kleinem Texturspeicher lädt die Grafikengine SGE automatisch kleinere Versionen einiger mitgelieferter Texturen. Um das Laden der Simulation zu beschleunigen und den Speicherbedarf von SILAB zu reduzieren, kann dies auch manuell aktiviert werden:

```
SILAB SGE {
    ForceTextureConfiguration = Small;
};
```

Bitte beachten Sie, dass dadurch die Qualität der grafischen Darstellung reduziert wird.

Manche Grafiktreiber weisen auch Fehler beim Laden von großen Texturen auf, obwohl dies laut der OpenGL-Hardware-Information unterstützt wird. In diesem Fall kann die Maximalgröße von 2D- und Cube-Map-Texturen insgesamt beschränkt werden:

```
SILAB SGE {
    MaxTextureSize=1024;
    MaxCubeTextureSize=1024;
};
```

Als Werte für die Maximalgröße sollten Zweierpotenzen gewählt werden.

2.2.1.1 Simulatorspezifische Texturen

Im Unterabschnitt `SGE SpecialTexture` der Konfigurationsdatei `SYSSGE.inc` kann die Zuweisung von wichtigen Texturen für die Simulation abgeändert werden. Das Format des Abschnitts ist:

```
SILAB SGE {
    SGE SpecialTexture {
        <Name> = texture.path.name;
        <Name> = "path/of/texture/file.ext";
    };
};
```

In den mitgelieferten 3D-Modellen werden mehrere Texturen durch einen frei gewählten `<Name>` vorgegeben, denen über diesen Abschnitt Dateien innerhalb des SILAB-Installationsverzeichnis zugewiesen werden. Die Dateien können entweder in der in Kap. 2.5 vorgestellten Notation (`texture.path.name`) oder direkt als Pfad- und Dateiname unterhalb des SILAB-Installationsverzeichnisses ("`path/of/texture/file.ext`") angegeben werden.

Welche Textur-`<Name>`n gesucht werden, können Sie in der Datei `\SILAB\lib\configurations\SGE\SGE_600.cfg` nachschlagen.

Hinweis: Bitte nehmen Sie keine Änderungen an der Datei

`\SILAB\lib\configurations\SGE\SGE_600.cfg` vor, da diese durch künftige SILAB-Updates überschrieben würden. Kopieren Sie die entsprechenden Zeilen stattdessen, wie oben beschrieben, nach

`D:\SILAB\lib\configurations\system\SYSSGE.inc`.

2.2.2 Grafikeinstellungen

Für die 3D-Grafikkarten auf den Grafikrechnern sollten einige Einstellungen vorgenommen werden, um die optimale Qualität und Leistung der Darstellung von SILAB zu erreichen.

Folgende Einstellungen werden empfohlen:

Bezeichnung bei NVidia	Bezeichnung bei ATI	Einstellung
Anti-Aliasing-Einstellungen	Smoothvision (Anti-Aliasing)	Anwendungsgesteuert
Anisotrope Filterung	Smoothvision (Anisotrope Filterung)	Anwendungsgesteuert
Systemleistung	Einstellungen für Textur	Qualität
Vertikale Synchronisierung	Vertikale Synchronisierung abwarten	Anwendungsgesteuert

In Abhängigkeit von Ihrem Grafikkarten-Hersteller und der installierten Treiberversion können Sie diese Einstellungen wie folgt durchführen:

2.2.3 NVidia, Aktuelle Treiberversionen

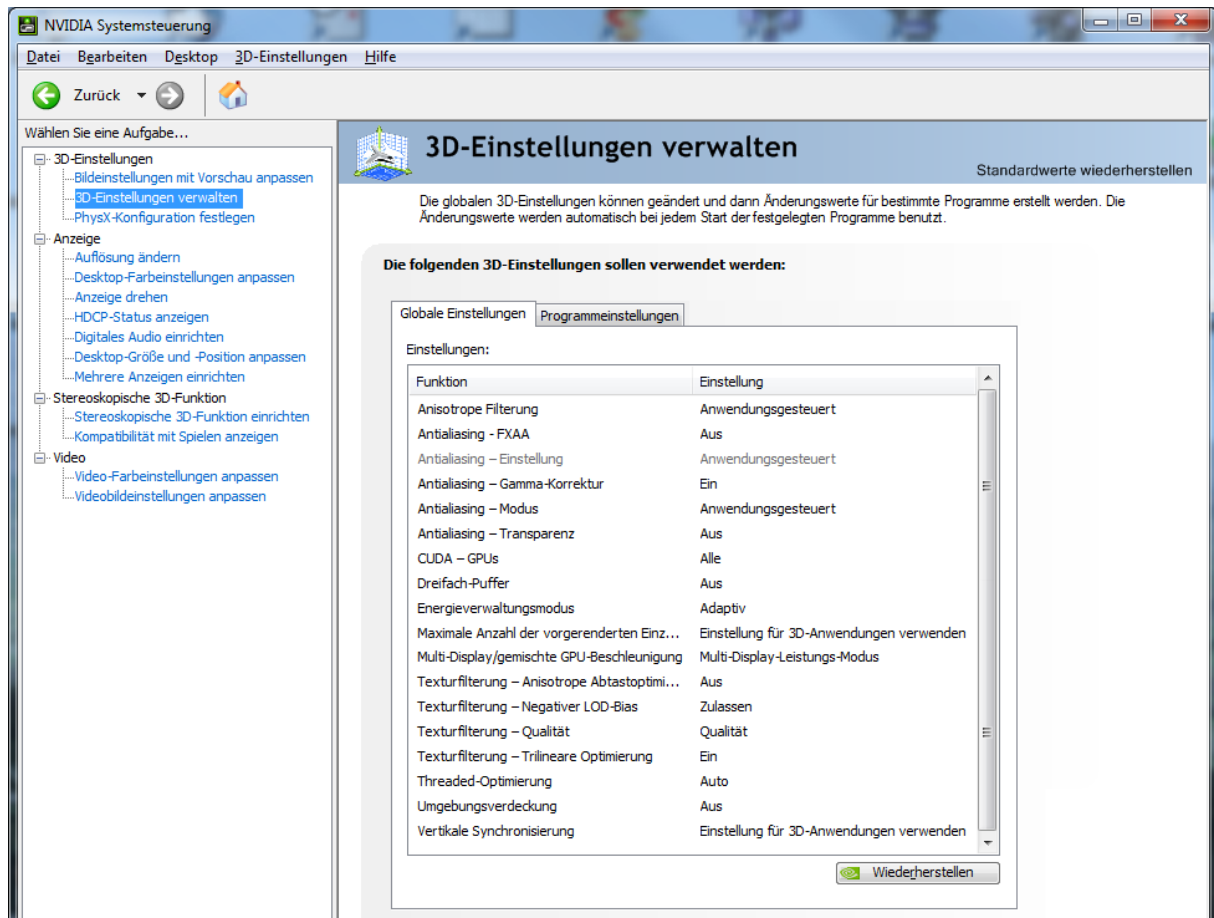
Rufen Sie zunächst das NVidia Control Panel auf:

Start → Einstellungen → Systemsteuerung → NVidia Control Panel

Zu den Leistungseinstellungen gelangen Sie über:

3D-Einstellungen → 3D-Einstellungen verwalten

Nehmen Sie folgende Einstellungen vor:



2.2.4 ATI, Catalyst-Treiberversionen

Rufen Sie zunächst das Catalyst-Kontrollzentrum auf:

(Mausklick mit rechter Taste auf den Desktop) → ATI Catalyst Kontrollzentrum

Nehmen Sie im Abschnitt *Spiele* → *3D-Anwendungseinstellungen* folgende 3D-Einstellungen vor:



Catalyst Control Center

Suchen

- > Angeheftet
- > Voreinstellungen
- > Desktop-Management
- > Allgemeine Anzeigeeinstellungen
- > AMD Eyefinity™ für mehrere Anzeigen
- > Meine digitalen Flachbildschirme
- > Meine VGA-Anzeigen
- > Video
- > Spiele
 - 3D-Anwendungseinstellungen
 - Anwendungsprofile
 - > Leistung
 - > Informationen

3D-Anwendungseinstellungen

Konfigurieren Sie Einstellungen zum Steuern der Bildqualität und Leistung von 3D-Anwendungen.

Anwendungsprofile

Sie können die Bildqualität und Leistung bestimmter Anwendungen anpassen, indem Sie Anwendungsprofile erstellen oder aktualisieren. Beim Erstellen bzw. Aktualisieren eines Profils werden 3D und AMD CrossFire™ Moduseinstellungen für alle Grafikprozessoren im Profil gespeichert.

<<Neue Anwendung>> Speichern

SMOOTHVISION HD: Anti-Aliasing

☒ Anwendungseinstellungen verwenden
☐ Morphologische Filterung

Stufe Slider 2X
Filter Box Beispiele: 2X

SMOOTHVISION HD: Anisotropische Filterung

☒ Anwendungseinstellungen verwenden

Pixelgenaue Muster Slider 2X

Tessellation

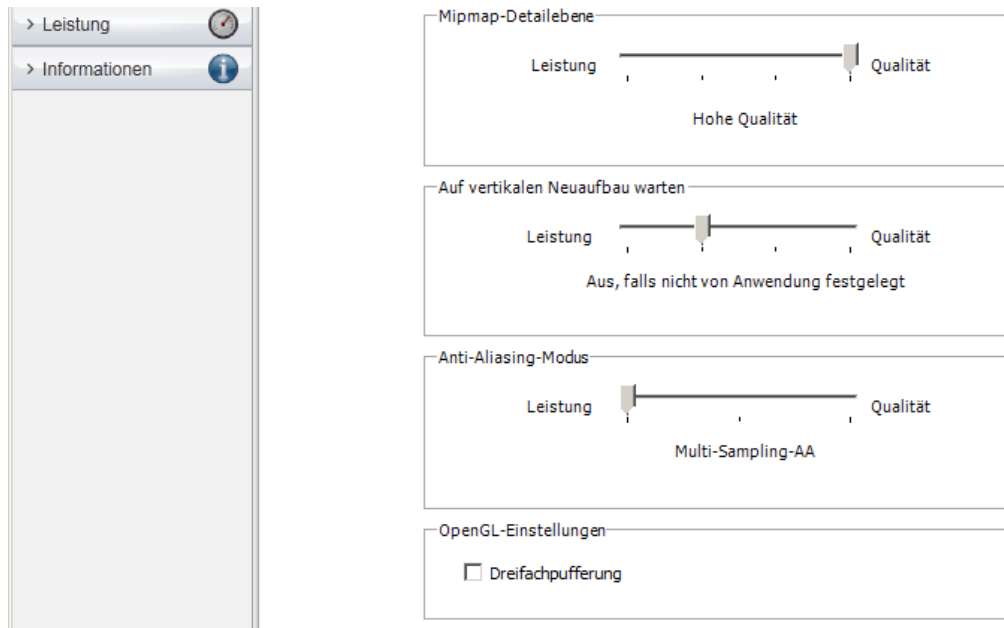
☒ AMD optimiert
☐ Anwendungseinstellungen verwenden

Ebene Slider

Catalyst A.I.

Qualität der Texturfilterung Slider Qualität

☒ Oberflächenformatoptimierung aktivieren



2.3 Threads

Die Darstellung der Szene in den SGEViews kann in eigene Threads verlagert werden. Dies empfiehlt sich insbesondere bei Multiprozessormaschinen (z.B. Dual-Core Rechner), um damit die Prozessoren des Grafikrechners möglichst gleichmäßig auszulasten.

Es sollten niemals mehr Threads benutzt werden, als physikalische Prozessoren vorhanden sind. Die Hyperthreading-Funktion mancher Intel-Prozessoren, mit der ein einziger Prozessor mehrere logische Kerne vorgaukelt, wirkt für SILAB nicht leistungssteigernd. Da sie die korrekte Verteilung der Threads auf die Prozessoren behindert, sollte sie abgeschaltet werden. Dies muss meist im BIOS des Grafikrechners durchgeführt werden.

Zur Anwendung von SGE-Threads muss lediglich der Parameter *Threaded* des zugehörigen DPUSGEView auf 1 gesetzt werden.

2.4 Kamera

Während der Simulation arbeiten die Datenbasis und die Fahrdynamik jeweils mit einer Referenzposition. Üblicherweise ist dies der Schwerpunkt des simulierten Fahrzeugs.

Diese Position ist normalerweise durch die Ausgänge

X , Y und Z	für die 3D-Position des Schwerpunkts im absoluten Koordinatensystem von SILAB ² und
<i>yaw</i> , <i>pitch</i> und <i>roll</i>	für die Drehung um die drei Koordinatenachsen

der Fahrdynamik (z.B. DPUmstVeda) festgelegt.

² d.h. die Entfernung in den Koordinatenachsen vom Startpunkt der Simulation, denn dieser wird als Ursprung des Koordinatensystems verwendet. Lesen Sie in Kap. 3.2.3 mehr über die SILAB-Koordinatensysteme.

Von der Datenbasis SCNX wird die Höhe des Fahrzeugschwerpunkts angepasst und als Ausgang *ZOut* zur Verfügung gestellt. Zusätzlich wird die Straßenneigung *pitchRoad* und *rollRoad* berechnet.

Die Position der „Kamera“ der Ansichten kann aber nicht mit dieser Fahrzeugposition gleichgesetzt werden. Für die Frontansichten der Simulation wird stattdessen normalerweise die Position des Kopfs des Fahrers verwendet, für Rückspiegel eine Position etwas vor dem Spiegel. Deswegen kann die Kameraposition jeder *SGEView*- und *SGERecorder*-Ansicht unabhängig voneinander eingestellt werden.

Normalerweise wird sie sich in der Nähe der Referenzposition befinden und sich mit dieser mitbewegen. Daher wird die Referenzposition über die Eingänge *X*, *Y*, *Z*, *yaw*, *pitch*, *roll*, *pitchRoad* und *rollRoad* der *DPUSGEView* bzw. *DPUSGERecorder* eingegeben. Die Position der Kamera wird dann relativ zu dieser Referenzposition durch die Parameter

CamA, *CamB* und *CamS* für eine Verschiebung gegenüber der Referenzposition und

CamBeta, *CamGamma* für zusätzliche Drehungen und *CamDelta* festgelegt.

Die Funktionsweise dieser Parameter wird in Abbildung 2 und in den Dokumenten zu *DPUSGEView* und *DPUSGERecorder* näher erläutert.

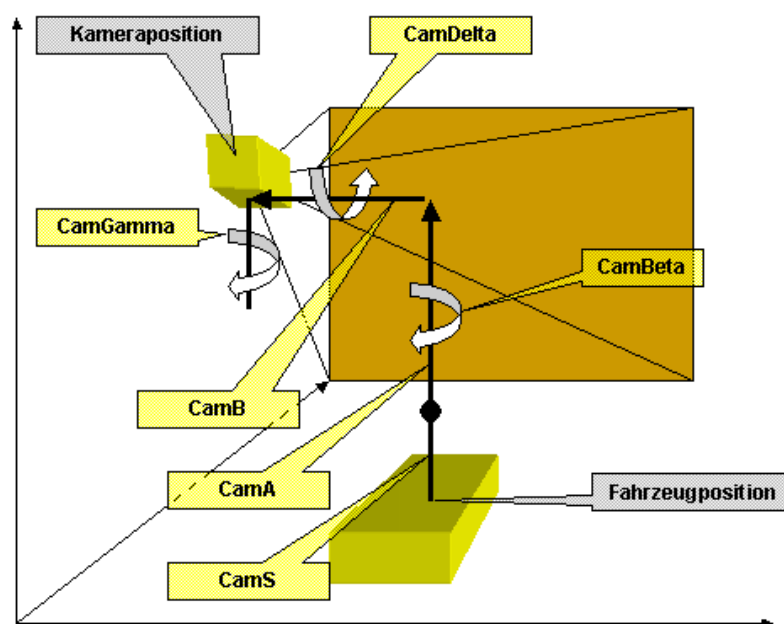


Abbildung 2 Kameraparameter für *DPUSGEView* und *DPUSGERecorder*

2.5 Verzeichnisstruktur

Die grafischen Objekte in der Visualisierung werden durch die geometrischen Informationen (Modelle) und durch Bilder der Oberflächen (Textures) repräsentiert.

Modelle werden im SIG- bzw. SGE-Format abgespeichert, ein SILAB-spezifisches Dateiformat, das im Kap. 3 näher erläutert wird.

Die Texturen werden als Bilddateien in den Standardformaten TGA (Targa Image Format) oder, seit Version 2.0 auch PNG (Portable Network Graphics) und JPG (Joint Picture Experts Group) eingelesen.

Alle Dateinamen, die für Modelle und Texturen zur Verwendung in der SGeworld benötigt werden, werden mit der betriebsystemunabhängigen Punkt-Notation angegeben:

`<Verzeichnis>.<Verzeichnis>. ... .<Datei>`

Der angegebene Dateiname wird entsprechend dem Kontext der Angabe um eine Präfixverzeichnisstruktur und eine Dateinamenerweiterung ergänzt und dann über das Betriebssystem geladen.

Die SGE-Bibliothek greift auf die folgende Verzeichnisstruktur zurück, in der die benötigten Objektdateien angeordnet werden:

```
\SILAB\lib\graphics\default
    Hauptverzeichnis der SGE Dateiverwaltung

... \models
    Unterverzeichnisse für Objektbeschreibungen
... \texture
    Unterverzeichnisse für Texturen
... \mipmap
    Unterverzeichnisse für MipMap-Texturen
... \shaders
    Unterverzeichnisse für Shader
```

Ein blauer Golf wird z.B. mit dem Namen `obstacles.Golf.blue` geladen. Diese Angabe wird unter dem Betriebssystem Windows erweitert zu

`\SILAB\lib\graphics\default\models\obstacles\Golf\blue.sig` und geladen.

Die in diesem Objekt enthaltene Textur `cars.parts.rad` wird erweitert zu

`\SILAB\lib\graphics\default\texture\cars\parts\rad.tga` und ebenfalls geladen.

In der SILAB Version 2.0 wurden weitere Verzeichnisse eingeführt, in denen Modelle und Texturen abgelegt werden können. Auch können nun die Namen einiger Verzeichnisse geändert werden. Dazu können im SILAB System-Abschnitt folgende Angaben durchgeführt werden:

```
SILAB System {
    PathODB = defaultname;
    PathODBAternate = alternatename;
};
```

Wenn die Variable `PathODB` angegeben wurde, wird anstelle vom Verzeichnis

`\SILAB\lib\graphics\default`
das Verzeichnis

`\SILAB\lib\graphics\defaultname`
verwendet. Die Modelle und Texturen werden also in einem komplett unterschiedlichen Ressourcenpfad gesucht.

Zusätzlich kann nun aber auch ein „Alternate Path“ verwendet werden, der zusätzlich zum Standard-Pfad durchsucht wird. Eine mögliche Anwendung des Alternate Path ist, einen ganzen Satz von Texturen für die Simulation unterschiedlicher Jahreszeiten auszutauschen. Dazu wird die Variable PathODBAternate im SILAB-System-Abschnitt genutzt. Wenn diese gesetzt ist, werden Objekte und Texturen zunächst im Verzeichnis

`\SILAB\lib\graphics\alternatename\models`
bzw.

`\SILAB\lib\graphics\alternatename\texture`
gesucht, bevor das Standardverzeichnis (`defaultname`) durchsucht wird.

Zusätzlich können projektspezifische Modell- und Texturpfade verwendet werden.

Für das Projekt `\SILAB\Projects\Demo`
können Modelle und Texturen auch unter

`\SILAB\Projects\Demo`
abgelegt werden.

2.6 Textur-Cache

Seit SILAB Version 2.0 werden, falls dies von der Grafikkarte unterstützt wird und die entsprechende Option in DPUSGEWorld nicht deaktiviert ist, komprimierte Texturen verwendet. Dies ermöglicht, bis zu viermal so viele Texturen in der Simulation zu laden bzw. höher aufgelöste Bilder zu verwenden. Um den Ladevorgang der Simulation zu beschleunigen, werden die komprimierten Texturen auf der Festplatte zwischengespeichert, und zwar im Verzeichnis `\SILAB\cache\texture`.

Dieser Cache wird automatisch neu erzeugt, wenn die ursprüngliche Bilddatei geändert wurde (z.B. bei einem Update einiger Grafikobjekte). Daher ist es normalerweise nicht nötig, die Cache-Dateien zu bearbeiten.

Sollten doch einmal Probleme auftreten (z.B. fehlende oder veraltete Texturen) kann das Cache-Verzeichnis aber jederzeit problemlos gelöscht werden, es wird dann beim nächsten Start der Simulation neu angelegt.

2.7 Taktung der Rechner

Durch unterschiedliche Taktraten der an der Simulation beteiligten Rechner und Ausgabegeräte kann es zu unruhigem Bildaufbau mit einzelnen „Rucklern“ kommen, selbst wenn keiner der Rechner überlastet ist, also die Simulation ohne Zeitverzögerungen durchgeführt werden kann.

Diese „Ruckler“ können durch eine sorgfältige Anpassung der Taktfrequenzen der Simulationsrechner minimiert werden. Dabei muss als Grundlage die Bildwiederholfrequenz der Ausgabegeräte gewählt werden. Für TFT-Projektoren oder LCD-Bildschirme beträgt diese im Allgemeinen 60 Hz. Als Taktfrequenzen für Simulationsrechner sollten nur ganzzahlige Vielfache dieser Frequenz eingestellt werden. Dabei muss die Taktfrequenz des Hauptrechners, auf dem die Simulation von Verkehr und Fahrdynamik durchgeführt wird, allerdings mindestens 100 Hz betragen.

Im Beispiel des in Kap. 2.1.2 vorgestellten Simulators empfiehlt sich folgende Computerkonfiguration:

```
Pool Simulator
{
    Executable = true;

    # Einbindung des Operators...

    Computer MAIN
    {
        IP = "...";
        Frequency = 120;
    };

    Computer VISLEFT
    {
        IP = "...";
        Frequency = 60;
    };

    # Genauso: VISFRONT, VISRIGHT
};
```

Wenn die Datenaufzeichnung der Simulation (die ebenfalls auf dem Hauptrechner stattfindet) mit einer anderen Taktfrequenz (z.B. 100 Hz) stattfinden soll, müssen entweder Ausgabegeräte beschafft werden, die einen Teiler dieser Frequenz als Bildwiederholfrequenz darstellen können (z.B. 50 Hz oder 100 Hz) oder kleinere Unruhmigkeiten beim Bildaufbau akzeptiert werden.

2.8 Geometrieverzerrung

Wenn für das Anzeigesystem eines Simulators Projektoren eingesetzt werden, muss abhängig vom Aufbau der Leinwände und der Position der Projektoren das ausgegebene Bild verzerrt werden, damit es aus der Sicht des Fahrers korrekt erscheint.

Besonders bei zylindrischen oder sphärischen Leinwänden ist eine Geometrieverzerrung unvermeidbar.

Seit SILAB Version 3.0 kann eine Geometrieverzerrung direkt von der Visualisierung SGE durchgeführt werden, so dass keine zusätzliche Hardware mehr benötigt wird.

Dazu wird die Szenerie zunächst in einen nicht sichtbaren Teil des Grafikspeichers gerendert. Das fertige Bild wird dann auf eine Verzerrungsgeometrie projiziert, die beliebig innerhalb des Ausgabefensters verschoben bzw. angeordnet werden kann.

Durch das Einbinden einer der DPUs, deren Name mit DPUSGEDistortion beginnt, wird dieser Algorithmus aktiviert.

Die Verzerrungsgeometrie kann über die Eingänge der DPU konfiguriert werden. Sie kann, auch während die Simulation läuft, angepasst werden. Die speziellen Eingänge der DPUs werden in den zugehörigen DPU-Dokumentationen beschrieben.

Folgende Verzerrungs-DPUs stehen zur Verfügung:

DPU	Geometrie
DPUSGEDistortionDummy	Es wird keine Verzerrung ausgeführt. Allerdings wird die Szene – wie bei allen Verzerrungs-DPUs – durch einen Offscreen-Puffer geleitet, was für die Erstellung von Videos und Screenshots vorteilhaft sein kann.
DPUSGEDistortionQuad	Ein Viereck, dessen Eckpunkte frei gewählt werden können. Hiermit kann z.B. eine Trapezverzerrung oder eine Drehung des Bilds durchgeführt werden.
DPUSGEDistortionBezier	Ein Bezier-Patch, d.h. eine Fläche, die durch eine zweidimensionale Bezier-Interpolation entsteht. Die Form wird über 4x4 Kontrollpunkte festgelegt. Die DPU eignet sich zur Projektion auf zylindrische oder sphärische Leinwände.
DPUSGEDistortionBezier5	Genau wie bei der DPUSGEDistortionBezier wird ein zweidimensionales Bezier-Patch verwendet. Hier wird die Form über 5x5 Kontrollpunkte festgelegt.
DPUSGEDistortionBezier5Ext	Wie bei der DPUSGEDistortionBezier5 wird ein zweidimensionales Bezier-Patch mit 5x5 Kontrollpunkten verwendet. Zusätzlich ist es hier möglich, mit Hilfe der DPUSGEDistortionBezier-Params die Verzerrung während der Laufzeit der Simulation anzupassen.

Durch den Einsatz von hardwarebeschleunigten OpenGL-Funktionen reduziert sich beim Einsatz der Geometrieverzerrung die erreichbare Framerate (FPS) nur um 5% bis 10%.

2.9 Weitere Konfiguration der Visualisierung

Noch mehr Konfigurationsoptionen für die Visualisierung und großen Einfluss auf die Darstellung des Szenarios bieten die „SCNX Params“ bzw. „SCNXSGE Params“. Diese sind in der „Referenz Datenbasis SCNX“ beschrieben.

3 ERWEITERUNG UM GRAFISCHE OBJEKTE

Fast alles, was in einer Ansicht (SGEView) der SILAB-Visualisierung zu sehen ist – beispielsweise die Landschaft, Straße, Fahrzeuge, Bäume und die Berge im Hintergrund, sind grafische Objekte. Ein grafisches Objekt besteht dabei zunächst aus der Beschreibung seiner geometrischen Struktur, die als Textdatei im SILAB-eigenen SGE-Objektformat abgelegt ist. Auf der Oberfläche des Objekts werden meist Bilddateien abgebildet (Texturen), die in den Standard-Dateiformaten TGA, PNG oder JPG abgelegt sein können. Außerdem wird eine Animation benötigt, durch die Bewegung und Verhalten des Objekts in der Simulation gesteuert werden können. Mit SILAB werden bereits Animationen mitgeliefert, durch die Objekte frei in der Szene platziert werden können (siehe auch DPUSGEOObject), wobei die Form des Objekts aber erhalten bleibt. Die Entwicklung weitergehender Animationen ist derzeit für SILAB-Anwender nicht möglich.

Die mitgelieferten und die selbst entworfenen Objekte können mit dem Programm SGEObjectPrev, das sich im Lieferumfang von SILAB befindet, betrachtet werden. Es ermöglicht auch den Import von grafischen Objekten aus anderen 3D-Modeller-Programmen im Wavefront-OBJ-Format und im FBX-Format. Näheres zu dieser Anwendung findet sich in Kapitel 4.

Die Visualisierung SGE ist abwärtskompatibel und kann auch grafische Objekte im älteren SIG-Objektformat einlesen, das von der Visualisierung SIG verwendet wurde. Die Objektformate SIG und SGE ähneln sich sehr.

3.1 Objekttypen

Generell kann zwischen zwei Typen grafischer Objekte unterschieden werden:

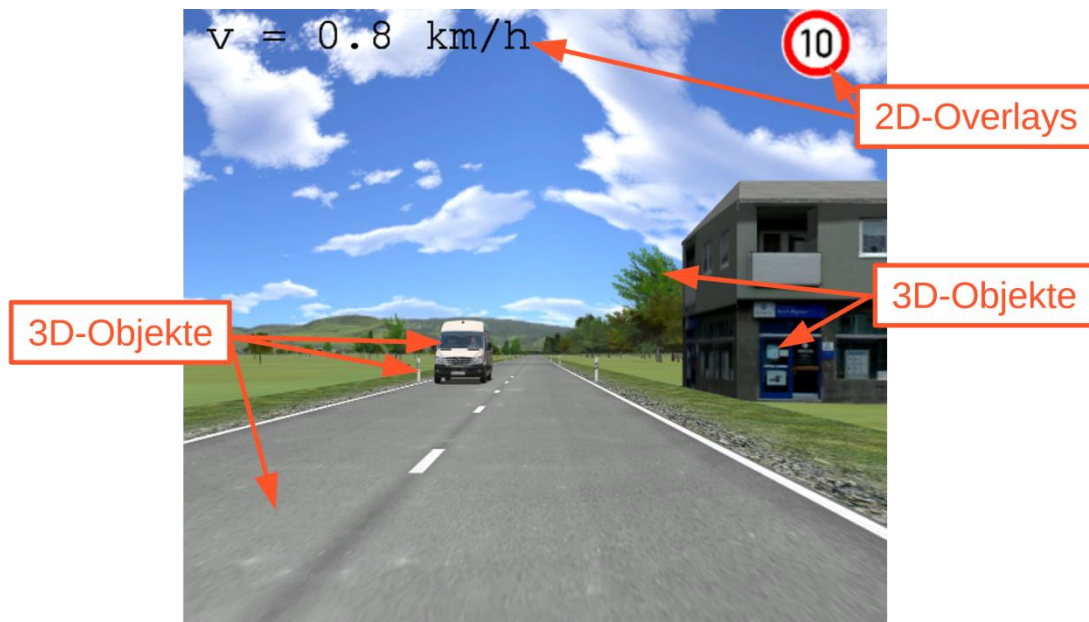


Abbildung 3 Objekttypen in der Visualisierung

3D-Objekte bilden die 3-dimensionale Umweltdarstellung.

2D-Overlayobjekte werden auf der Ebene des Bildschirms dargestellt und überlagern die Visualisierung der Umweltdarstellung.

3.2 Grundlagen zum Objektentwurf

3.2.1 Einteilung in Unterobjekte

Mit der SGE-Bibliothek können 3D-Objekte mit einer (theoretisch) unbegrenzten Zahl von Unterobjekten realisiert werden. Einem Objekt bzw. Unterobjekt kann jeweils nur eine Textur oder Mipmap zugeordnet werden. Die Einteilung eines Objektes in Unterobjekte erfolgt zunächst rein inhaltlich. Werden jedoch unterschiedliche Texturen in einem Objekt benötigt, dann muss die Aufteilung des Objekts in Unterobjekte mit jeweils einer Textur erfolgen. Eine rein inhaltliche Aufteilung eines Objekts in Unterobjekte ist z.B. bei den vier Rädern eines Fahrzeugs gegeben. Jedes Rad wird zwar durch die gleiche Textur und eventuell dem gleichen Vertex-Satz³ definiert, die vier Räder eines Fahrzeugs haben jedoch unterschiedliche Positionen am Fahrzeug.

3.2.2 Vertextabelle und Primitive

Jedes Objekt und Unterobjekt enthält eine Vertex-Tabelle, in der die Vektoren, die Texturkoordinaten und eventuell vorhandene Farben bzw. Normalenvektoren zu einem Vertex zusammengefasst werden. Doch die Vertex-Tabelle allein bildet noch keine dreidimensionale Objektstruktur. Das sichtbare Objekt besteht aus Primitiven wie texturierten Dreiecken, Vierecken oder Polygonen. Diese Primitive werden in einer Primitivendefinition zusammengefasst. Ein Primitiv referenziert dabei eine Liste von Vertices aus der Vertex-Tabelle über den Index des Vertex. Dies hat den Vorteil, dass viele der definierten Vertex-Einträge von mehreren Primitiven gemeinsam benutzt werden können.

3.2.3 Koordinatensysteme

In der SGE-Bibliothek werden für 3D-Objekte und 2D-Overlays unterschiedliche Koordinatensysteme verwendet:

3.2.3.1 3D-Koordinatensystem

Der Ursprung des 3D-Koordinatensystems befindet sich bei dem Aufsetzpunkt des Ego-Fahrzeugs zum Zeitpunkt des Simulationsstarts auf der Referenzspur der Strecke⁴. Die positive X-Achse zeigt zu diesem Zeitpunkt nach vorne, die positive Y-Achse nach rechts und die positive Z-Achse nach oben. Die Vektoren haben die Einheit Meter. Positive Winkel (in rad) entlang der (positiven) Achse sind linksdrehend.

³ Ein Vertex fasst eine Texturkoordinate, einen Vektor und eventuell noch einen Normalenvektor bzw. eine Farbe zusammen.

⁴ Das Ego-Fahrzeug kann beim Start mit einem Offset zu diesem Punkt aufgesetzt werden.

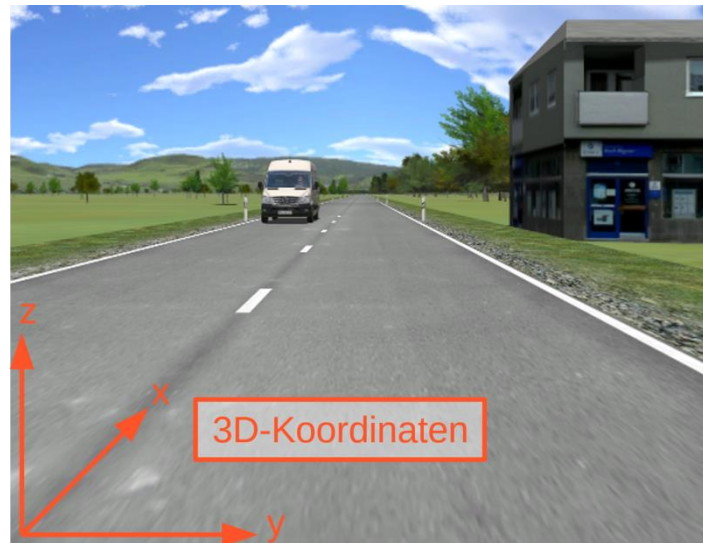


Abbildung 4 3D-Objektkoordinatensystem

3.2.3.2 2D- Koordinatensystem

Der Ursprung dieses Koordinatensystems befindet sich in der linken, oberen Ecke des Bildschirms. Die positive X-Achse zeigt nach rechts und die positive Y-Achse nach unten. Die Z-Achse kann als Schichtung der Objekte angesehen werden. Positionsangaben in den Vertices haben die Einheit Bildschirmpixel. Positive Winkel (in rad) entlang der (positiven) Achse sind linksdrehend.



Abbildung 5 2D-Overlay-Koordinatensystem

3.2.4 Texturen und Mipmaps

Ein Vertex enthält zusätzlich zum dreidimensionalen Vektor seiner Position eine Texturkoordinate. Die Textur wird über das entstehende Primitiv entsprechend der Texturkoordinate und des Vektors gespannt und in der im Vertex angegebene Farbe eingefärbt. Wird im Vertex keine Farbe definiert, dann wird die Textur in der globalen Objektfarbe eingefärbt.

Je nach Abstand des texturierten Primitives zur Kameraposition muss die Textur vergrößert oder verkleinert werden. Dies führt bei stark strukturierten Texturen in der Regel zu einer gewissen Unruhe bei zunehmender Entfernung zur Kameraposition. Diese Unruhe kann durch die Verwendung von Mipmaps anstelle von Texturen vermindert werden. Eine Mipmap enthält eine Tabelle von Texturen unterschiedlicher Größe. Mit zunehmender Entfernung zur Kamera werden immer kleinere Texturen benutzt. Diese kleineren Texturen werden üblicherweise automatisch beim Laden der

Textur durch Skalierung des Bildes mit Mittelung erzeugt und reduzieren dadurch die unruhige Darstellung.

3.2.4.1 Texturgrößen

Die Textur (egal ob eine reine Textur oder als Teil einer Mipmap) ist immer rechteckig und kann im Allgemeinen nur *Zweierpotenzen* als Kantenlänge⁵ annehmen. Da alle Texturen von allen in der Simulation verwendeten Objekten im Speicher der Grafikkarte gehalten werden müssen, sollte darauf geachtet werden, dass die Größe einer Textur so klein wie möglich, also nur so groß wie nötig, gehalten wird.

Moderne Grafikkarten unterstützen unter gewissen Voraussetzungen auch Texturen mit Kantenlängen, die keine Zweierpotenzen sind. In den meisten Fällen funktioniert dies aber nur, wenn Mipmapping deaktiviert ist und sollte daher normalerweise beim Objektentwurf nicht ausgenutzt werden.

3.2.4.2 Texturen

Texturen und Mipmaps werden aus Bildern im TGA- und PNG-Format mit 24 Bit Farbtiefe eingelesen. Zusätzlich kann ein 8 Bit Alpha-Kanal enthalten sein, der von SGE als Transparenzinformation genutzt wird. Der Wert 0 (Schwarz) im Alpha-Kanal wird dabei als vollkommen transparent interpretiert und der Wert 255 (Weiß) als undurchsichtig.

Das JPG-Format wird ebenfalls unterstützt, es ist allerdings weniger zu empfehlen, da es verlustbehaftet komprimiert und keinen Alpha-Kanal unterstützt. JPG-Texturen sind also immer undurchsichtig.



Abbildung 6 Teilweise transparente 32-Bit Textur. Zur Verdeutlichung ist der Alpha-Kanal (in Bildbearbeitungsprogrammen häufig als „Maske“ bezeichnet) nochmals separat dargestellt.

⁵ 1,2,4,6,8,16,32,64,128,256,512,1024 oder 2048 Pixel. Die maximale Größe ist auf 2048x2048 beschränkt.

3.2.4.3 Texturfilter

Eine Textur wird durch den Rendervorgang vergrößert oder verkleinert dargestellt. Die Art und Weise, wie die einzelnen Pixel einer Textur bei der Vergrößerung bzw. Verkleinerung zusammengefasst werden, wird über den Texturfilter bestimmt.

Nearest-Filter:

Der Nearest-Filter wählt den zum Zentrum des berechneten Pixels am nächsten gelegenen Pixel der Textur aus.

Linear-Filter:

Der Linear-Filter berechnet den gewichteten Durchschnitt jener vier Pixel der Textur, die dem Zentrum des berechneten Pixels am nächsten liegen.

Bei modernen Grafikkarten gibt es keine Leistungsunterschiede zwischen dem Nearest- und dem Linear-Filter, so dass im Allgemeinen der Linear-Filter bevorzugt werden sollte.

Zusätzlich zu diesen Standard-Filtern bieten moderne Grafikkarten die Möglichkeit, Texturen anisotrop zu filtern. Die Darstellung von Texturen, die auf perspektivisch sehr „schräg“ stehende Flächen (z.B. die Straßenoberfläche) projiziert werden, wird dadurch erheblich verbessert. Anisotrope Filter haben jedoch einen deutlich erhöhten Rechenaufwand. Um diesen Filtertyp in SGE-Objekten zu nutzen, müssen folgende Voraussetzungen erfüllt sein:

1. Die Grafikkarte muss anisotrope Texturfilterung unterstützen
2. Die Grafikkarte muss so eingestellt sein, dass Anwendungen die anisotrope Texturfilterung kontrollieren dürfen
3. In der Instanz von DPUSGEView, die für das entsprechende Ausgabefenster zuständig ist, muss der Parameter „EnableTextureAnisotropy“ auf 1 und die Parameter „MinTextureAnisotropy“, „MedTextureAnisotropy“ und „MaxTextureAnisotropy“ auf den gewünschten Grad der Filterung gestellt werden (vgl. Dokument „DPUSGEView“).

Unter diesen Voraussetzungen ist die anisotrope Filterung aktiviert. Einzelne SGE-Objekte können bei Bedarf den Grad der Filterung individuell verändern bzw. die Filterung abschalten. Dies erfolgt über die Zuweisung

TextureAnisotropy = <Grad>;

Für <Grad> sind Werte von 0 bis 3 möglich, wobei Grad 0 den anisotropischen Filter deaktiviert und höhere Werte zu schöneren Filterungs-Ergebnissen führen.

3.2.4.4 Texturkoordinatensystem

Die Texturkoordinaten werden als S und T bezeichnet und relativ zur Texturgröße definiert. Die linke untere Ecke einer Textur bildet den Ursprung [0.0, 0.0] des Texturkoordinatensystems und die rechte obere Ecke bildet den Punkt [1.0, 1.0].



Abbildung 7 Textur-Koordinatensystem

Eine Texturkoordinate ist nicht auf den Wertebereich $[0.0 \dots 1.0]$ beschränkt. Die Behandlung der Texturkoordinaten die aus diesem Wertebereich hinausgehen, können für jede Achse (S,T) getrennt im Texturfilter festgelegt werden.

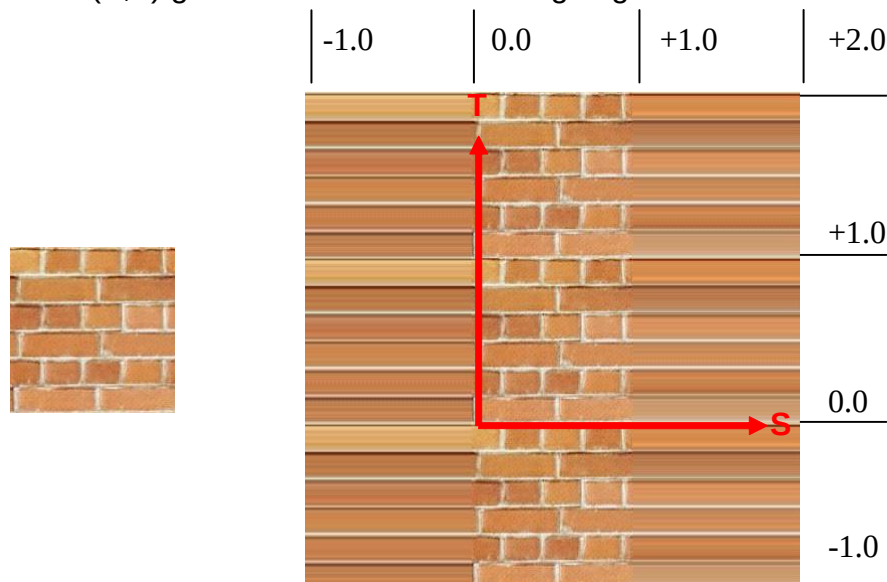


Abbildung 8 Beispiel für **ClampS | RepeatT**

In Abbildung 8 wird die Originaltextur (im Bild links zu sehen) auf ein Quadrat mit Texturkoordinaten im Bereich von $[-1.0 \dots +2.0]$ aufgezogen. Die Texturfiltereinstellungen sind auf **ClampS** (keine Wiederholung entlang der S-Achse) und **RepeatT** (unendliche Wiederholung entlang der T-Achse) gesetzt. Dabei wird die Originaltextur entlang der T-Achse zyklisch wiederholt, bis die Texturkoordinate erreicht ist. Bei der S-Achse sorgt die Filtereinstellung **ClampS** dafür, dass die letzte Pixelreihe so lange wiederholt wird, bis die Texturkoordinate erreicht wird.

3.3 Formale Objektdefinition

3.3.1 Syntax der Objektdateien

SGE-Objektdateien sind Plain-Text-Dateien, die am besten mit einem Editor für allgemeine Programmiersprachen wie Geany, TextPad, oder Crimson Editor bearbeitet werden können.

Hinweis: Für den frei verwendbaren Editor Geany stellen wir in der Datei (*D:\SILAB\System\Syntax\SILAB_Syntax_Geany.zip*) eine Syntaxhervorhebungsdefinition zur Verfügung. Die Konfiguration des Editors wird in der *Installationsanleitung SILAB 7.1*, Abschnitt *Einrichten eines Texteditors*, beschrieben.

Bei der Schreibweise von SGE-Objektdateien wird zwischen **Groß- und Kleinschreibung** unterschieden. **Leerzeichen** und **Tabulatoren** sind für die Syntax nicht relevant, es wird aber empfohlen, die Zeilen konsistent einzurücken, um den Überblick zu behalten.

Wie in C++ können **Kommentare** in die Objektdateien eingefügt werden:

- Die Zeichenfolge `//` markiert den Anfang eines Zeilenkommentars; alles bis zum Ende dieser Zeile wird ignoriert.
- Die Zeichenfolge `/*` markiert den Anfang eines mehrzeiligen Kommentars; alles bis zum Ende `*/` wird ignoriert.

3.3.2 Objektdefinition

Grammatik 1 Definition eines SGE-Objektes

```
define SGE3DObject <Name> (0...1)<Erweiterung>
{
    (0...1)<Texturdefinition>
    (0...*)<Parameter>
    (0...1)<Cookies>
    (0...1)<Shader-Parameter>
    (0...1)<Benutzerdefinierte Parameter>
    (0...*)<Vertextabelle>
    (0...*)<LOD-Primitive>
    (0...*)<Subobjekt>
};

<Erweiterung> →
    : <Objektname>
```

Für jedes Objekt kann maximal eine Textur (bzw. ein Textursatz bei Multitexturing) festgelegt werden, die auf alle Flächen des Objekts projiziert wird. Wenn keine Textur angegeben ist, wird die Oberfläche mit der angegebenen Farbe (siehe Abschnitt 3.3.3.5) gefüllt. Die Geometrie des Objekts wird über eine Vertex-Tabelle, in der Vektoren zusammen mit Texturkoordinaten und Farben definiert werden und eine Definition der Primitive festgelegt. Die Primitive können auch entfallen. In diesem Fall kapselt das Objekt nur seine Subobjekte, die Parameter werden dann (weitgehend) igno-

riert. Jedes Objekt kann zahlreiche Subobjekte besitzen. Zusätzlich können bestimmte Eigenschaften eines Objektes über verschiedene Parameter eingestellt werden.

Optional kann ein Objekt ein anderes erweitern.

Als zu erweiterndes Objekt kommt ein beliebiges anderes SIG-Objekt in Frage. Der *<Objektname>* wird als Ressourcenname angegeben (siehe Abschnitt 2.5).

Eine Erweiterung bewirkt, dass das neu definierte Objekt alle Parameter, die Geometrie und die Subobjekte vom erweiterten Objekt übernimmt.

Nachträglich können aber im neu definierten Objekt noch eine *<Texturdefinition>* vorgenommen werden sowie *<Parameter>* und *<Cookies>* angegeben werden. Die Veränderung der Geometrie und das Hinzufügen zusätzlicher Subobjekte ist in einem Objekt, das ein anderes erweitert, aber nicht zulässig.

3.3.3 Parameter

Grammatik 2 Parameter

<Parameter> →
<Token> = *<Argument>*;

<Argument> →
<Token> |
<Zahl> |
<Farbe> |
<Vektor> |
<OR-Flag> |
<Tokenliste>

<Tokenliste> →
 (.)*<Token>*

<OR-Flag> →
 (|)*<Token>*

3.3.3.1 Animation

Animation = *<Name>*;

Eine Animation wird für die Positionierung und Bewegung des Objekts in der Simulation benötigt. Die Animation des Hauptobjekts positioniert dabei auch alle Subobjekte (und deren Subobjekte), für die keine eigene Animation angegeben ist. Viele Animationen können nur in einem Hauptobjekt ausgeführt werden, d.h. falls diese als Animations-Parameter im Subobjekt angegeben werden, wird dies ignoriert.

Die SGE 3D-Engine enthält bereits einige allgemein verwendbare Animationen, die im Folgenden vorgestellt werden. Die einzelnen Visualisierungen (z.B. TRFSGE, MOPSGE) enthalten spezielle Animationen (beispielsweise für

Fahrzeuge), die nur für die Objekte dieser Visualisierung verwendet werden können.

Animation: „default“

Die Animation positioniert das Objekt und seine Subobjekte an einer festen Position in der Simulation. Diese Position wird durch die „Translate“, „Rotate“, und „Scale“ Parameter des Objekts und der Subobjekte festgelegt.

Um ein (Sub-)Objekt O zu positionieren, wird wie folgt vorgegangen:

- Für alle Objekte in der Hierarchie vom Hauptobjekt hinunter zum Objekt O wird ausgeführt:
 - Das Objekt wird mit dem im Scale-Parameter angegebenen Wert skaliert.
 - Das Objekt wird mit den im Rotate-Parameter angegebenen Werten um die X-, Y- und Z-Achse gedreht.
 - Das Objekt wird um die im Translate-Parameter angegebene Distanz verschoben.

Die Animation „default“ wird auch verwendet, wenn keine Animation im Objekt angegeben wird. Die Standardwerte der Parameter Translate, Rotate und Scale sind aber so gewählt, dass keine Transformation des Objekts durchgeführt wird.

Animation: „multidefault“

Diese Animation wird zur Darstellung von Objekten verwendet, die üblicherweise in großen Mengen verwendet werden, wobei jedes einzelne der Objekte relativ einfach ist⁶. Die Animation funktioniert genau wie die default-Animation. Sie ermöglicht es aber der DPUSCNXSGE, mehrere dieser Objekte gemeinsam zu behandeln und somit effizienter darzustellen.

Animation: „billboard“

Diese Animation dient der Visualisierung von Billboard-Objekten.

Billboard-Objekte sind Fotos von realen Objekten, die auf Rechtecke „gespannt“ sind und die während der Simulation immer senkrecht zur Blickrichtung des Fahrers gedreht werden.

Die Animation geht davon aus, dass jeweils vier aufeinander folgende Vektoren eines Objekts (oder seiner Subobjekte) ein solches Rechteck bilden und dreht dieses dann in Richtung des Fahrers. Dabei wird als Zentrum der Drehung die Position des Objekts verwendet, welche im Translate-Parameter angegeben wurde. Die häufigste Anwendung für Billboards sind Bäume.

Animation: „multibillboard“

Diese Animation dient genau wie „billboard“ der Visualisierung von Billboard-Objekten. Der einzige Unterschied ist, dass jedes Rechteck des Objekts um seinen Schwerpunkt gedreht wird und damit als unabhängiges Billboard-Objekt interpretiert wird. Im Gegensatz zur „billboard“-Animation können also

⁶ Dies bedeutet, dass die Objekte typischerweise keine Subobjekte besitzen und aus einer geringen Anzahl (< 50) von Vertices bestehen. Beispiele hierfür sind Pylonen und Leitpfosten.

mit der Multibillboard-Animation mehrere unabhängige Flächen mit einem einzigen Objekt dargestellt werden, die allerdings alle nur aus je einem einzigen Rechteck bestehen dürfen.

Animation: „field“

Diese Animation dient zur Modellierung von Objekten, die sich der Landschaftsoberfläche anpassen, wie z.B. Feldern und Straßen. Zunächst werden alle Transformationen genau wie bei der „default“-Animation ausgeführt.

Anschließend wird jeder Punkt des Objekts noch an die Landschaftsoberfläche angepasst: Die Animation positioniert jeden Punkt des Objekts (mit Koordinate $\langle x, y, z \rangle$) auf der Position $\langle x, y, \text{Landschaftshöhe}(x,y)+z \rangle$.

Mit der field-Animation animierte Objekte müssen als Speichertyp **Storage=Unique** setzen, damit jeder Vektor separat transformiert werden kann.

Animation: „rotor“

Die Animation ist für sich kontinuierlich drehende Objekte geeignet. Im Objekt muss dazu der Animationstyp **AnimationAlways** angegeben werden.

Translationen und Skalierungen werden wie bei der default-Animation ausgeführt.

Über Cookies (siehe Abs. 3.3.4) kann jedoch eine kontinuierliche Rotation des Objekts um die Achsen des (lokalen Objekt-)Koordinatensystems erzielt werden:

`PeriodX` = *<Zeit für eine volle Umdrehung in ms>;*

`OffsetX` = *<Rotationswinkel beim Start der Simulation>;*

Falls ein negativer Wert für `OffsetX` gewählt wird, wird das Offset für jedes Objekt dieses Objekttyps zufällig gewählt.

Analog hierzu können die Cookies `PeriodY` und `OffsetY` für Rotationen um die Y-Achse bzw. `PeriodZ` und `OffsetZ` für Rotationen um die Z-Achse verwendet werden.

AnimationType = <Token>;

Behandlung der Animation. Prinzipiell können einer Animation verschiedene Animationstypen zugewiesen werden. Einige Animationen überschreiben jedoch den Animationstyp des Objektes, damit sie korrekt arbeiten können.

Token	Beschreibung
AnimationAlways	Wird in jedem Renderschritt animiert
AnimationCameraSync	Wird synchron zur Kameraposition animiert.
AnimationOneshot	Wird nur einmal beim Einfügen animiert
AnimationTriggered	Wird nur neu animiert, wenn ein internes Ereignis ausgelöst wird (z.B. in SCNXSGE: Veränderung der Landschaft durch neu hinzukommendes Straßenstück)

Für die Animation *default* sollte stets der Animationstyp **AnimationOneshot** verwendet werden, bei der Animation *billboard* muss hingegen **AnimationAlways** verwendet werden.

3.3.3.2 Boundingbox

Boundingbox = (X1,Y1,X2,Y2,X3,Y3,X4,Y4);

Die Boundingbox für die ODB-Schnittstelle⁷. Dieser Parameter muss nicht angegeben werden, normalerweise wird für die Boundingbox das Minimum und Maximum der Vektoren des Objekts verwendet. In manchen Fällen entsteht dadurch aber eine zu große Boundingbox, beispielsweise bei Bäumen: Hier wird automatisch die komplette Fläche der Baumkrone berechnet. Eine Kollision soll aber nur festgestellt werden, wenn der Testfahrer den Stamm berührt. Daher wird die Größe des Baumes für ODB mit Hilfe des Boundingbox-Parameters auf 1 m in X- und Y-Richtung beschränkt:

```
Boundingbox = (0.5,-0.5,0.5,0.5,-0.5,0.5,-0.5,-0.5);
```

Dieser Parameter muss, falls er angegeben wird, *nach* der Vertex-Tabelle stehen.

BoundingboxScale = <Zahl>;

BoundingboxScale = (X,Y);

Alternativ zur festen Vorgabe der Boundingbox-Größe kann diese auch als Vielfaches der berechneten Fläche (d.h. Minimum und Maximum der Vektoren) festgelegt werden. Es kann entweder ein Skalierungswert in X- und Y-Richtung oder ein einheitlicher Skalierungswert für beide Achsen angegeben werden. Mit folgender Angabe ist die Größe eines Baumes für ODB gleich der halben Fläche der Baumkrone:

```
BoundingboxScale = 0.5;
```

3.3.3.3 Typ

Type = ((),<Zahl>);

Typkodierung für die ODB-Schnittstelle. Das Tupel muss mindestens aus drei Einträgen bestehen. Alle in der Simulation vorhandenen Objekte werden in Objekttypen eingeteilt, damit bei einer Abfrage von ODB leicht bestimmte Arten von Objekten (z.B. Fahrzeuge oder Bäume) unterschiedlich behandelt werden können. Eine Liste der Objekttypen findet sich im Dokument „Referenz Objektdatenbank ODB“.

3.3.3.4 Transparenz

AlphaTest = <Zahl ∈ (0.0,1.0)>;

Legt die Behandlung der transparenten Anteile einer Textur fest. Liegt der Alpha-Wert einer Textur unter dem angegebenen Wert, dann wird dieses Pixel vollständig transparent gezeichnet. Falls *AlphaTest* = 0 angegeben wird, werden alle Pixel gezeichnet.

AlphaTest = [<positive Zahl>, <positive Zahl>;

Wie zuvor, allerdings werden separate Werte festgelegt, abhängig davon wie der Wert des Eingangs *TransparencyMultisample* der *DPUSGEView* geschaltet ist.

⁷ s. „Handbuch SILAB Enterprise“ und „Referenz Objektdatenbank ODB“

Falls *TransparencyMultisample=false*, wird der erste angegebene Wert verwendet.

Falls *TransparencyMultisample=true*, wird der zweite angegebene Wert verwendet.

Beide Werte müssen größer als 0 sein. Üblicherweise wird der zweite Wert kleiner als der erste gewählt, um einen weichen Übergang an der Objektkante zu erhalten, z.B. **AlphaTest = [0.5, 0.2]**;

3.3.3.5 Objektfarbe

Color = <Farbe>;

Globale Farbe des Objektes. Jeder Vertex, dem keine eigene Farbe zugewiesen wird, bekommt diese Farbe. Die Farbwerte werden als Tupel (**R,G,B,A**) im Bereich von 0 bis 1 angegeben, siehe Grammatik 11 Farbe, S. 44.

3.3.3.6 Culling

Culling = CCW | CW | OFF;

Bei abgeschaltetem Culling wird jedes Primitiv gezeichnet. Wird das Culling auf CW gesetzt, dann werden nur Primitive gezeichnet, deren Vektoren im Uhrzeigersinn definiert wurden. Bei dem Wert CCW gilt das gleiche gegen den Uhrzeigersinn.

Der Standardwert ist **Culling = CCW**.

Für *undurchsichtige Objekte mit einer geschlossenen Außenhaut* sollte aus Effizienzgründen das Culling aktiviert werden, es muss dabei nur darauf geachtet werden, dass die Vektoren aller Primitive in derselben Reihenfolge (im Uhrzeigersinn / gegen Uhrzeigersinn) angegeben werden.

Für *beleuchtete Objekte (RenderClass=ZBuffered)* sollte auf jeden Fall das Culling aktiviert werden, da sonst nur die Primitive, die im Gegenuhrzeigersinn definiert wurden, korrekt beleuchtet werden.

Für *2D-Objekte (RenderClass=Overlay)* sollte das Culling deaktiviert werden, da das Objekt sonst beim Aktivieren des *Mirror*-Modus unsichtbar wird.

3.3.3.7 Linienbreite

LineWidth = <Zahl>;

Gibt die Breite einer Linie in Pixeln an. Dies betrifft nur Primitive vom Typ Lines, LineLoop oder LineStrip (siehe Kap. 3.3.8). Nicht-ganzzahlige Werte werden nur dann berücksichtigt, wenn Antialiasing aktiviert ist. Der Standardwert ist **LineWidth = 1**.

3.3.3.8 Tiefenpufferbehandlung

DepthTest = <-1 oder +1>;

Wenn für DepthTest -1 angegeben wird, wird der Tiefenpuffer-Test abgeschaltet, das Objekt wird dann über alle vorher gezeichneten Objekte gezeichnet, selbst wenn es hinter ihnen liegt. Für den Wert +1 (Standardwert) wird der Tie-

fenpuffer-Test angeschaltet. Somit wird das Objekt korrekt durch andere, vor ihm liegende Objekte verdeckt.

DepthWrite = *<Bool>*;

Wenn für DepthWrite **false** angegeben wird, wird das Objekt nicht in den Tiefenpuffer gezeichnet, d.h. nachfolgende Objekte werden dann über dieses Objekt gezeichnet, auch wenn sie eigentlich hinter ihm liegen. Dies ist nur in Sonderfällen sinnvoll (z.B. Nebel- oder Rauch-Objekte).

Der Standardwert **true** führt dazu, dass das Objekt in den Tiefenpuffer gezeichnet wird und es somit nur von den nachfolgenden Objekten verdeckt wird, die vor ihm liegen.

3.3.3.9 Nebelbehandlung

Fog = *<Zahl $\in$ (0.0,1.0)>*;

Globaler Multiplikator des Nebels, durch den die Wirkung des Nebels auf dieses Objekt abgeschwächt werden kann. Mit dem Standardwert 1.0 wirkt sich der Nebel vollständig auf das Objekt aus, mit 0.0 ignoriert das Objekt den Nebel. Die Rückleuchten eines Fahrzeugs erhalten z.B. den Nebelwert 0.4, damit diese im Nebel noch sichtbar sind, obwohl das Fahrzeug selbst schon im Nebel verschwunden ist.

3.3.3.10 Translation / Skalierung

Translate = *<Vektor>*;

Translationsvektor für dieses (Sub-)Objekt. Nicht alle Animationen unterstützen die Angabe eines Translationsvektors. Die Animation *default* berücksichtigt diesen Wert.

Rotate = *<Vektor>*;

Rotationsvektor für dieses (Sub-)Objekt. Nicht alle Animationen unterstützen die Angabe eines Rotationsvektors. Die Animation *default* berücksichtigt diesen Wert.

Scale = *<Vektor>*;

Skalierungsfaktor als Vektorangabe für dieses (Sub-)Objekt. Nicht alle Animationen unterstützen die Angabe eines Skalierungsfaktors. Die Animation *default* berücksichtigt diesen Wert.

3.3.3.11 Renderklasse

RenderClass = *<Token>*;

Die Renderklasse definiert die Behandlung des (Sub-)Objekts. In jedem Darstellungsschritt werden die Renderklassen in folgender Reihenfolge abgearbeitet:

Token	Beschreibung
NotZBuffered	3D-Objekt, das nicht in die Z-Pufferbehandlung eingefügt wird.
ZBuffered	3D-Objekt, das in die Z-Pufferbehandlung eingefügt wird. (Die meisten Objekte gehören zu dieser Renderklasse.)
Transparent	3D-Objekt, das nach allen nicht transparenten Objekten gerendert wird.
NotZBufferedOverlay	Overlay-Objekt, das nicht in die Z-Pufferbehandlung eingefügt wird.
Overlay	Overlay-Objekte. Die Vektoren werden unabhängig von der Kameraposition als Bildschirmkoordinaten interpretiert und dem gerenderten Bild überlagert. Die Z-Koordinate wird entsprechend einer Schichtung der Objekte interpretiert.
LightOverlay	Spezielle Objekte für globale Lichter im Nachtsicht-Modus (z.B. Sonnenlicht)
Light	Spezielle Objekte für lokale Lichter im Nachtsicht-Modus (z.B. Objekt für die Lichtquelle einer Straßenlampe)
LightResolve	Spezielle Objekte für Antialiasing-Operationen im Nachtsicht-Modus.

RenderID = *<Zahl>*;

Definiert die Reihenfolge von Subobjekten, die zur Renderklasse **Transparent** gehören. Bei der Zahl muss es sich um einen nicht-negativen, ganzzahligen Wert handeln. Subobjekte mit höherer **RenderID** werden zuerst gezeichnet. Der Standardwert ist 0.

3.3.3.12 Shading

ShadeMode = **Smooth** | **Flat**;

Der Schattierungsmodus bestimmt, wie die Farbe eines Primitivs berechnet wird, wenn seine Eckpunkte unterschiedliche Farben besitzen.

Beim Schattierungsmodus **Flat** erhält das ganze Objekt die Farbe seines ersten Eckpunkts.

Beim Schattierungsmodus **Smooth** wird zwischen den Farben der Eckpunkte interpoliert, es wird ein Farbverlauf erzeugt.

3.3.3.13 Sichtbarkeit

Show = *<Bool>*;

Flag für die Sichtbarkeit. Wird ein (Sub-)Objekt unsichtbar gemacht, dann kann die Animation dieses (Sub-)Objekt wieder sichtbar machen und umkehrt.

3.3.3.14 Sichtbarkeitstest

VisTest = *<Zahl>*;

Die Visualisierung SGE enthält einen einfachen Sichtbarkeitstest-Algorithmus zur Optimierung der Darstellung. Dieser basiert auf einem Frustum-Culling-Verfahren, d.h. alle Objekte, die vollständig außerhalb des Sichtbereichs der Kamera liegen, werden nicht dargestellt.

Aus Rechenzeitgründen kann der Sichtbarkeitstest nur mit einer vereinfachten Geometrie des Objekts durchgeführt werden. Hierzu wird eine Kugel um den Ursprung des lokalen Koordinatensystems des Objekts verwendet. Der Radius der Kugel wird durch die *<Zahl>* des *VisTest*-Parameters definiert. Wenn der *VisTest*-Parameter nicht angegeben ist, wird das angegebene Objekt immer gezeichnet.

Durch die Verwendung des *VisTest* können in vielen Situationen etwa 20-30% mehr Objekte dargestellt werden.

3.3.3.15 Level of Detail

LodDistance = [*(,)<Zahl>*];

Die Visualisierung SGE unterstützt die Darstellung mehrerer unterschiedlich komplexer Varianten des Objekts, abhängig von dessen Abstand zur Kamera. Diese werden als Levels of Detail bezeichnet. LOD 0 ist das detaillierteste Objekt, LOD 1, 2, usw. sind dann immer größere Versionen des Objekts. In Kap. 3.3.8 wird die Definition von Levels of Detail behandelt.

Der erste Eintrag des LodDistance-Arrays gibt an, bis zu welcher Entfernung (in Metern) LOD 0 dargestellt wird. Der zweite Eintrag definiert dann den Höchstabstand für LOD 1 usw. Insgesamt muss also immer genau ein Eintrag weniger als die Anzahl definierter LOD angegeben werden.

Hinweis: Die Darstellung von Levels of Detail basiert auf teilweise auf dem Algorithmus für Sichtbarkeitstests. Daher muss immer zusätzlich der **VisTest**-Parameter (siehe Kap. 3.3.3.14) angegeben werden, damit sie korrekt funktioniert.

3.3.3.16 Vertex-Spezifikation

TangentSpace = *<Bool>*;

Falls *true* angegeben wird, berechnet die Visualisierung beim Laden des Objekts automatisch Vertextangenten und -bitangenten für alle angegebenen Vektoren. Diese Tangenten werden für manche Shader benötigt, insbesondere wenn Normal Mapping verwendet wird.

Der Standardwert ist *false*, was bedeutet, dass entweder keine Tangentenvektoren benötigt werden oder dass sie explizit angegeben werden (dies wird in Kapitel 3.3.7.4 beschrieben).

3.3.3.17 Texturen und Mipmaps

Einem (Sub-)Objekt kann maximal eine Textur („Bild“) zugeordnet werden. Eine Ausnahme davon bildet lediglich das Multitexturing (siehe Abs. 3.3.3.18), bei dem mehrere Texturen übereinander projiziert werden. Dies ist aber deutlich rechenaufwändiger und nur in Spezialfällen sinnvoll.

Grammatik 3 Textur

<Texturdefinition>→
Texture = (,)<Token>;
 <Parameter>

Texture = <Dateiname>;

Der Texturname wird relativ zum SGE-Grafikverzeichnis angegeben (siehe Kap. 2.5).

TextureFilter = <OR-Flag>;

Der Parameter Texturfilter fasst die Eigenschaften ‚**Filterfunktion**‘, ‚**Texturgenerierung**‘ und ‚**Texturkoordinatenbehandlung**‘ zusammen. Die einzelnen Einstellungen müssen mit dem OR-Zeichen | kombiniert werden.

Filterfunktion:

Eine Textur wird durch den Rendervorgang verzerrt. Die Texturfilter legen fest, in welcher Weise die Textur vergrößert bzw. verkleinert wird.

Token	Beschreibung
MagLinear	Vergrößerung der Textur mit dem Linear-Filter
MagNearest	Vergrößerung der Textur mit dem Nearest-Filter
MinLinear	Verkleinerung der Textur mit dem Linear-Filter
MinNearest	Verkleinerung der Textur mit dem Nearest-Filter
MinLinearMipMapLinear	Bei dieser Option werden die beiden Mipmaps ausgewählt, deren Pixel nach dem Linear-Filter am ehesten der berechneten Pixelgröße entspricht. Der endgültige Pixel wird durch den gewichteten Durchschnitt der beiden Pixel berechnet.
MinNearestMipMapLinear	Bei dieser Option werden die beiden Mipmaps ausgewählt, deren Pixel nach dem Nearest-Filter am ehesten der berechneten Pixelgröße entspricht. Der endgültige Pixel wird durch den gewichteten Durchschnitt der beiden Pixel berechnet.
MinLinearMipMapNearest	Bei dieser Option wird diejenige Mipmap ausgewählt, deren Pixel nach dem Linear-Filter am ehesten der berechneten Pixelgröße entspricht.
MinNearestMipMapNearest	Bei dieser Option wird diejenige Mipmap ausgewählt, deren Pixel nach dem Nearest-Filter am ehesten der Pixelgröße entspricht.

Nearest-Filter:

Der Nearest Filter wählt den zum Zentrum des berechneten Pixel am nächsten gelegenen Pixel (nach der Manhattan-Metrik) der Textur aus.

Linear-Filter:

Der Linear-Filter berechnet den gewichteten Durchschnitt von den vier Pixeln der Textur, die dem zum Zentrum des berechneten Pixels am nächsten liegen.

Texturgenerierung:

Mit diesem Parameter wird festgelegt, wie die letztendlich in der Simulation dargestellte Textur erzeugt wird:

Token	Beschreibung
Texture	Die Textur wird als reine Textur aus dem Texturpfad geladen
MipMap	Die Textur wird als Mipmap aus dem Mipmappfad geladen
TextureMipMap	Aus der Textur wird automatisch eine Mipmap generiert

Texturkoordinatenbehandlung:

Token	Beschreibung
ClampS	Keine Wiederholung entlang der S-Achse. Für Koordinaten außerhalb von [0,1] wird die letzte Pixelreihe des Texturbilds verwendet.
ClampT	Keine Wiederholung entlang der T-Achse. Für Koordinaten außerhalb von [0,1] wird die letzte Pixelreihe des Texturbilds verwendet.
ClampToBorderS	Keine Wiederholung entlang der S-Achse. Für Koordinaten außerhalb von [0,1] wird die Randfarbe (TextureBorderColor) verwendet.
ClampToBorderT	Keine Wiederholung entlang der T-Achse. Für Koordinaten außerhalb von [0,1] wird die Randfarbe (TextureBorderColor) verwendet.
RepeatS	Unendliche Wiederholung entlang der S-Achse.
RepeatT	Unendliche Wiederholung entlang der T-Achse.

Vordefinierte Filterkombinationen:

Die wichtigsten Filterkombinationen sind nochmals zu eigenen Tokens zusammengefasst.

Token	Beschreibung
StdTextureFilter	MinLinearMipMapNearest MinLinear MagLinear
FineTextureFilter	MinLinearMipMapLinear MinLinear MagLinear
StdRepeat	RepeatS RepeatT StdTextureFilter
StdClamp	ClampS ClampT StdTextureFilter
StdRepeatS	RepeatS ClampT StdTextureFilter
StdRepeatT	ClampS RepeatT StdTextureFilter
FineRepeat	RepeatS RepeatT FineTextureFilter
FineClamp	ClampS ClampT FineTextureFilter
FineRepeatS	RepeatS ClampT FineTextureFilter
FineRepeatT	ClampS RepeatT FineTextureFilter

TextureBaseLevel = <Zahl>;

TextureMaxLevel = <Zahl>;

TextureMaxLod = <Zahl>;

TextureMinLod = <Zahl>;

Bei der Verwendung von Mipmaps (TextureFilter = ... | Mipmap | ... ;) oder automatisch generierten Mipmaps (TextureFilter = ... | TextureMipmap | ... ;) wird mit diesen beiden Parametern die Anzahl der Mipmapstufen (TextureBaseLevel / TextureMaxLevel) und die Behandlung der Überblendung (TextureMinLod / TextureMaxLod) zur nächsten Stufe eingestellt.

TextureBorderColor = <Farbe>;

Randfarbe für die Textur. Diese wird nur mit dem Filtermodus *ClampToBorder* verwendet. Die Randfarbe wird genau wie beim Attribut *Color* durch Rot-, Grün-Blau- und Alpha-Werte im Bereich 0 bis 1 definiert.

TextureAnisotropy = <Zahl>;

Mit diesem Wert wird Grad der anisotropen Filterung eingestellt (siehe Abschnitt 3.2.4.3).

TextureMipMapStretchFilter = <Filterwert>;

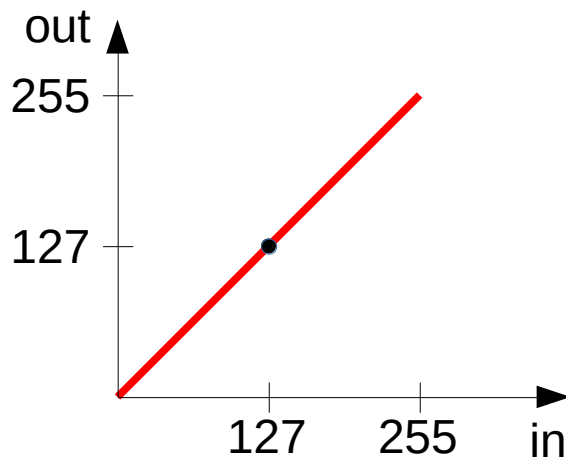
TextureMipMapShiftFilter = <Filterwert>;

Konfiguriert die Skalierung des Bilds bei der Generierung von Mipmaps, d.h. wenn **TextureFilter** = ... | **TextureMipMap** verwendet wird.

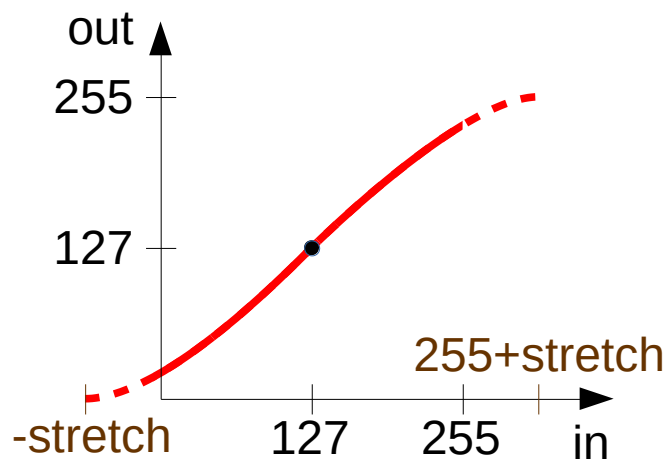
Falls der Parameter angegeben ist, wird bei jedem Verkleinerungsschritt zusätzlich ein Filter angewendet und die Farbwerte werden, separat für jeden Farbkanal, bearbeitet.

Der Filterwert wird als Tupel (**R,G,B,A**) im Bereich [-127 ... 127] angegeben, dies entspricht der Effektstärke. Wenn für einen Farbkanal der Wert 0 angegeben ist, wird dafür kein Filter angewendet.

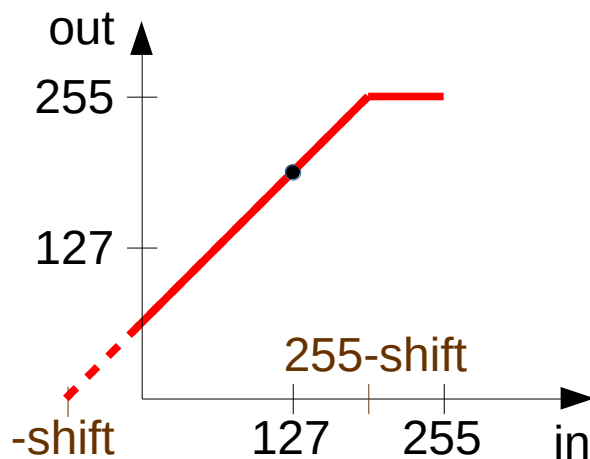
Für *stretch* = 0, *shift* = 0 wird keine Änderung am Farbwert vorgenommen:



Für *stretch* > 0 wird der Farbkanal in einer S-Kurve verzerrt. Das Bild wird flauer:



Für $shift > 0$ wird der Mittelpunkt des Farbkanals nach oben verschoben. Das Bild wird heller:



Analog hierzu wird das Bild für $shift < 0$ dunkler, und für $stretch < 0$ wird der Farbkanal zusammengestaucht und das Bild wird stärker gesättigt bzw. „knaackiger“. Selbstverständlich können beide Filterparameter auch kombiniert werden.

3.3.3.18 Multitexturing

Moderne Grafikkarten unterstützen die Anwendung mehrerer Texturen auf dasselbe Dreieck bzw. Primitiv, die gemäß einer in einem separaten Shader-Programm definierten Funktion kombiniert werden. Dies ist allerdings deutlich rechenaufwändiger als die Angabe einer einzelnen Textur und sollte daher nur für anderweitig nicht erzielbare Effekte eingesetzt werden.

Um Multitexturing zu verwenden, werden mehrere Texturen angegeben. Dazu dient der indizierte Textur- Parameter

Texture[i] = <Dateiname>;

mit dem eine Textur für die i-te Textureinheit der Grafikkarte definiert werden kann.

Der nicht indizierte **Texture**-Parameter definiert die Textur für die nullte Textureinheit

und ist daher äquivalent zu **Texture[0]**. Üblicherweise können maximal vier Texturunits, d.h. **Texture[0]** bis **Texture[3]** verwendet werden.

Für jede Textur können Texturfilter-, Randfarben-, Anisotropie- und Level-of-Detail-Einstellungen separat vorgenommen werden, dazu werden die indizierten Texturparameter **TextureFilter[i]**, **TextureAnisotropy[i]** usw. verwendet.

Um die Farbwerte mehrerer Texturen zu kombinieren, muss ein geeignetes Shaderprogramm angegeben werden (siehe Abs. 3.3.3.19).

3.3.3.19 Shader

Insbesondere bei der Verwendung von Multitexturing (siehe Abs. 3.3.3.18) muss definiert werden, wie die Farbwerte der Texturen und des Objekts zu einer endgültigen Farbe verrechnet werden. Dies wird in einem Shader-Programm angegeben.

Für jedes Objekt werden zwei Shaderprogramme verwendet, ein Vertex-Shader (der einmal pro Vertex des Objekts aufgerufen wird) und ein Pixel-Shader (der einmal für jeden gezeichneten Pixel aufgerufen wird).

Diese werden mit den Parametern

```
VertexShader = <Dateiname>;
```

```
PixelShader = <Dateiname>;
```

definiert.

Die Shader-Dateien werden dabei im Verzeichnis

```
\\SILAB\\lib\\graphics\\default\\shaders\\vertex bzw.
```

```
\\SILAB\\lib\\graphics\\default\\shaders\\pixel
```

gesucht. Für die Angabe von Dateinamen wird ansonsten dieselbe betriebssystemunabhängige Notation wie bei Texturen und Modellen verwendet (siehe Abs. 2.5).

Falls im Objekt keine speziellen Shader eingebunden werden, wählt SGE einen Standard-Shader für das Objekt aus. Als Vertex-Shader wird *FixTransform* verwendet. Als Pixel-Shader wird *Color* (falls keine Textur angegeben ist) oder *Tex0* (falls mindestens eine Textur angegeben ist) verwendet.

Zur Beschleunigung der Darstellung von Schatten können für die Schattenberechnung des Objekts vereinfachte Shader-Programme verwendet werden. Diese werden über die Parameter **DepthVertexShader** bzw. **DepthPixelShader** und **ShadowVertexShader** bzw. **ShadowPixelShader** definiert.

Folgende Standard-Shader werden mit SILAB mitgeliefert:

Standard-Vertex-Shader	Beschreibung
FixTransform	Normale Transformation der Vertices.
FixTransform_Reflect	Normale Transformation der Vertices für spiegelnde Objekte (z.B. Fensterscheiben).
Billboard	Ausrichtung aller Flächen zum Fahrer hin. Muss für alle (Sub-)Objekte mit Billboard-Animation verwendet werden.
MultiBillboard	Ausrichtung aller Flächen zum Fahrer hin. Muss für alle (Sub-)Objekte mit MultiBillboard-Animation verwendet werden.
PedTransform	Transformation einzelner Körperteile von Fußgängern. Muss für alle (Sub-)Objekte mit MOV-Animation verwendet werden.
CarTransform	Transformation von Fahrzeugen. Muss für alle (Sub-)Objekte mit Car-Animation verwendet werden.
CarTransform_Reflect	Transformation von spiegelnden Fahrzeugteilen (z.B. Karosserie, Fensterscheiben).

Standard-Pixel-Shader	Beschreibung
Color	Färbt das Objekt in der angegebenen Farbe (Color) ein.
ColorReflect0	Färbt das Objekt in der angegebenen Farbe (Color) ein und fügt Spiegelungen hinzu. Sollte für einfarbige spiegelnde Objekte (z.B. Fensterscheiben) verwendet werden. Als Texture[0] muss die Spiegeltextur angegeben werden (üblicherweise \$Environment).
Tex0	Färbt das Objekt mit der Farbe der Texture[0] ein.
Tex0Reflect1	Färbt das Objekt mit der Farbe der Texture[0] ein und fügt Spiegelungen hinzu. Sollte für texturierte spiegelnde Objekte (z.B. Karosserie) verwendet werden. Als Texture[1] muss die Spiegeltextur angegeben werden (üblicherweise \$Environment).

Die Definition eigener Shader-Programme durch SILAB-Anwender wird derzeit nicht unterstützt. Die mit SILAB mitgelieferten Standard-Shader haben jedoch Parameter, welche im Abschnitt Shader der Objektdefinition gesetzt werden können (siehe Abs. 3.3.5).

3.3.3.20 HeightInfoX (Beeinflussung des Höhenmodells)

Vom Anwender im Objects-Abschnitt einer Course-Typ-Definition (siehe „Referenz Datenbasis SCNX“) platzierte grafische Objekte können die Höhe der Landschaft unterhalb des Objekts beeinflussen.

Grammatik 4 HeightInfo

```

<HeightInfo>→
    HeightInfoX = [<NP>,
        <Y-Koord>, <XStart>, <XEnd>, (<Anzahl Stützpunkte>)<Infl>, <H>,
        ...
    ];

```

Die Definition der HeightInfo besteht aus einem Raster, das über eine (prinzipiell beliebig große) Anzahl von Linien aufgespannt wird. Die Linien sind parallel zur X-Achse des Objekts. Jede Linie hat gleich viele Stützpunkte <NP>.

Für jede Linie muss die Y-Koordinate <Y-Koord> angegeben werden, auf der sich die Linie befindet, sowie die X-Koordinaten des Start- und Endpunkts der Linie <XStart> und <XEnd>.

Die Linie wird dann durch <NP> Stützpunkte definiert, wobei für jeden Stützpunkt jeweils ein Einfluss-Faktor <Infl> und die Höhe der Landschaft relativ zur Z-Position

des Objekts *<H>* (in Metern) definiert werden muss. Der Einfluss-Faktor gibt an, wie stark das Objekt an dieser Stelle die Landschaft beeinflusst: 0 = gar nicht, 1 = sehr stark. (Zwischenwerte sind möglich.)

Derzeit gilt die Einschränkung, dass alle Linien eines HeightInfo ein Raster mit gleichmäßig großen, rechteckigen Zwischenräumen bilden müssen.

Das bedeutet: *<XStart>* und *<XEnd>* müssen für alle Linien eines HeightInfo gleich sein und die Differenz von *<Y-Koord>* von zwei aufeinanderfolgenden Linien muss jeweils gleich sein.

Es wird empfohlen, dass der Abstand zwischen benachbarten Rasterpunkten zwischen 5 und 50 m beträgt, um eine effiziente Umsetzung der HeightInfo durch das Landschaftsmodell zu ermöglichen.

Für die Rasterpunkte am Rand des Objekts sollte *<Inf>*=0 gelten (erste und letzte Zeile und Spalte des Rasters), andernfalls könnten die definierten Senken bzw. Täler aus dem Bereich des Objekts herauslappen.

3.3.4 Cookies

Grammatik 5 Cookies

```
<Cookies>→
    define Cookies
    {
        (0..*)<Cookie-Name> = <Cookie-Wert>;
    };
```

Über Cookies können bestimmte Parameter für die Animation gesetzt werden. Diese Parameter sind abhängig von der Animation. Die meisten SGE-Standard-Animationen (siehe Kap. 3.3.3.1) definieren keine Cookies, die Funktion wird aber von einigen Animationen der Visualisierungs-Module (z.B. TRFSGE) verwendet. Als *<Cookie-Name>* können beliebige Strings verwendet werden, als *<Cookie-Wert>* kommen nur Zahlen in Frage.

3.3.5 Shader-Parameter

Grammatik 6 Shader-Parameter

```
<Shader-Parameter>→
    Shader
    {
        (0..*)<Parameter-Name> = [( )<Zahl>];
    };
```

Über Shader-Parameter können die gesetzten Shader des Objekts konfiguriert werden. Ein Shaderparameter ist dabei entweder eine einzelne Zahl oder ein Array von Werten, die als Gleitkommazahlen angegeben werden. Die mit SILAB mitgelieferten Standard-Shader haben folgende Parameter, welche vom Anwender gesetzt werden können:

Parameter	Vorhanden in Shader	Beschreibung
OmniDirectionalSurface = (0..1);	(alle)	Definiert raue Flächen, die Teilflächen in mehrere Richtungen haben (z.B. Blätter an Bäumen). Standardwert ist 0 , Maximalwert ist 1 . Je höher der Wert, desto schwächer wird die Fläche abgeschattet, wenn das Sonnenlicht von hinten auf diese fällt.
SpecularPower = (0..128);	ColorReflect0, Tex0Reflect1	Bestimmt die Größe des Reflexionspunkts einer Lichtquelle auf spiegelnden Oberflächen. Je höher der Wert ist, desto kleiner ist der Punkt.
ReflectionAmount = [(0..1), (0..1)];	ColorReflect0, Tex0Reflect1	Die Angabe [a,b] bestimmt den Anteil des reflektierten Lichts. Normalerweise gilt 1 > a > b > 0 . Der Wert a wird verwendet, wenn der Betrachter tangential auf das Objekt blickt, der Wert b , wenn er senkrecht darauf blickt. Für Zwischenwerte wird linear interpoliert.

3.3.6 Benutzerdefinierte Parameter

Grammatik 7 Benutzerdefinierte Parameter

```

<Benutzerdefinierte Parameter>→
    User
    {
        (0..*)<Parameter-Name> = [(,)<Zahl>];
    };

```

Mit benutzerdefinierten Parametern können zusätzliche Informationen zum Objekt festgelegt werden. Diese Informationen werden von der SGE 3D-Engine ignoriert. Sie können allerdings von Animationen oder sonstigen Objekt-Bearbeitungswerkzeugen verwendet werden.

3.3.7 Vertextabelle

Grammatik 8 Vertextabelle

```

<Vertextabelle>→
    Vertices =
    {
        (,)<Vertex>
    }; |
    <Vertexliste>

<Vertex>→
    (<Vektor>) |
    (<Texturkoordinate>,<Vektor>)
    (<Texturkoordinate>,<Vektor>,<Farbe>)

```

In der Vertextabelle werden alle Eckpunkte des (Sub-)Objekts definiert. Diese können dann in den Primitivdefinitionen (siehe Kap. 3.3.8) verwendet werden, um die Geometrie des Objekts zu definieren. Dabei werden die Einträge der Vertextabelle

durch ihre Position in der Liste referenziert. Der erste Eintrag der Vertextabelle erhält die Nummer 0.

Für jeden Vertex muss die geometrische Position im Raum definiert werden, diese wird als Vektor bezeichnet. Optional kann zusätzlich auch ein Normalenvektor, eine Texturkoordinate und eine Farbe definiert werden.

Achtung: Für alle Vertices sollten die gleichen Informationen angegeben werden (d.h. entweder werden bei jedem Vertex Texturkoordinaten angegeben oder bei keiner). Ansonsten werden diese Informationen möglicherweise ignoriert.

3.3.7.1 Vektordefinition

Grammatik 9 Vektor

```
<Vektor>→
  <X,Y,Z|nX,nY,nZ> |
  <X,Y,Z> |
  <X,Y|nX,nY> |
  <X,Y>
```

Für jeden Vektor wird die Koordinate im dreidimensionalen Raum als X-, Y-, und Z-Wert angegeben. Die Koordinaten werden in spitze Klammern eingefasst.

Beispiel: $\langle 1, 2, -3.5 \rangle$

Zusätzlich kann noch ein Normalenvektor angegeben werden. Hiermit wird eine Richtung spezifiziert, die senkrecht auf der Oberfläche durch diesen Punkt steht.

Beispiel: $\langle 1, 2, -3.5 \mid 0, 1, 0 \rangle$

Der Normalenvektor wird zur Berechnung der Beleuchtung eines Objekts verwendet. Wird kein Normalenvektor angegeben, legt SGE diesen automatisch als senkrecht auf dem Punkt stehend fest.

Optional kann die Z-Koordinate weggelassen werden, was insbesondere für 2D-Overlayobjekte interessant ist. Der Z-Wert wird dann auf 0 gesetzt.

3.3.7.2 Texturkoordinate

Grammatik 10 Texturkoordinate

```
<Texturkoordinate>→
  [S,T]
```

Die Texturkoordinate wird als Vektor im Texturkoordinatensystem (siehe Abschnitt 3.2.4.4) angegeben und in eckige Klammern eingefasst.

Beispiel: $[0.1, 0.8]$

3.3.7.3 Farbvektor

Grammatik 11 Farbe

```
<Farbe>→
  (R,G,B,A)
```

Die Farbe wird als Farbtupel durch ihren Rot-, Grün-, Blau- und Alphawert angegeben. Dabei ist der Wertebereich von R, G, B und A jeweils 0..1. Sie wird durch runde Klammern eingefasst.

Beispiel: (1, 0, 0, 1) ist **Rot**, (0, 0.5, 0.5, 1) ein **dunkles Türkis**.

Der **Alphawert** bestimmt die Transparenz des Objekts. Der Wertebereich ist hier 0 = vollständig transparent bis 1 = komplett undurchsichtig.

3.3.7.4 Vertexliste

Grammatik 12 Vertexliste

```
<Vertexliste>→
(0..4)TV(0..1)N(0..1)G(0..1)B(0..1)C(0..4)U =
(
    (<Zahl>
);
```

Die Vertexliste bietet eine kompaktere und flexiblere, aber weniger gut lesbare Alternative zur Vertextabelle.

Grundlegend wird dabei zunächst das Format der Vertexliste definiert (durch Angabe der enthaltenen Informationen für jede Vertex) und danach werden die Werte der einzelnen Vertices einfach in einer kommaseparierten Liste aufgezählt.

Der Formatspezifikator („TTVNGBCUU“) gibt an, welche Daten für jede Vertex vorhanden sind. Dabei sind folgende Daten möglich:

T	eine Texturkoordinate (S,T) (es können 0-4 Texturkoordinaten angegeben werden)
V	die Vertexkoordinaten (X,Y,Z) (diese müssen angegeben werden)
N	die Vertexnormalen (nx,ny,nz) (werden automatisch berechnet, falls weggelassen)
G	die Vertextangenten (tx,ty,tz) (werden automatisch berechnet, falls weggelassen)
B	die Vertexbitangenten (bx,by,bz) (werden automatisch berechnet, falls weggelassen)
C	die Vertexfarbe (R,G,B,A) (kann weggelassen werden, falls alle Vertices dieselbe Farbe haben sollen)
U	zusätzliche Userdata für die Animation (ux,uy,uz,uw) (es können 0-4 Sätze von Userdaten angegeben werden)

Die Werte aller Vertices werden dann einfach in dieser Reihenfolge angegeben. Ein Beispiel:

```
TV( # s      t      x      y      z      # Vertex
    0.922444, 0.331241, -0.0463608, -2.12194, 7.65484, # Vertex 0
    0.9995,   0.999499, -0.0613608, -2.1998,  7.7176,  # Vertex 1
    0.000499487,0.62416, -0.0613608, -1.19042, 7.68235, # Vertex 2
    0.0421364, 0.000493973, -0.0463608, -1.23248, 7.62378 # Vertex 3
);
```

3.3.8 Primitive

Grammatik 13 Definition eines SGE-Objektes

```

<LOD-Primitive> →
    Primitives[<Zahl>] =
    {
        (0...*) <Primitiv>
    }; |
    <Primitivliste>

```

```

<Primitiv> →
    <Primitivtyp>
    {
        (,) <Zahl>
    };

```

Mit den Primitiven wird die Oberfläche des Objekts definiert. Die Oberfläche kann dabei aus Dreiecken, Vierecken oder Polygonen zusammengesetzt sein. Die möglichen Primitivtypen sind in der Tabelle 1 aufgezählt.

Jeder Primitivtyp wird als eine Folge ganzer Zahlen beschrieben, die jeweils Einträge der Vertextabelle (d.h. die Ecken des Objekts) referenzieren. Dabei wird ab 0 (für die erste Ecke) gezählt.

Beispiel: Das Primitiv

```

Quads {
    0,1,2,3, 4,6,8,9
};

```

erzeugt zwei Vierecke, wobei das erste zwischen den Eckpunkten 0,1,2 und 3 der Vertextabelle aufgespannt wird und das zweite zwischen den Eckpunkten 4,6,8 und 9. Beachten Sie bitte immer, dass alle Flächen (egal ob Dreiecke, Vierecke oder Polygone) stets eben und konvex sein müssen, da sie sonst evtl. nicht korrekt dargestellt werden können.

Ein Objekt kann aus mehreren Primitiven bestehen, die alle zusammen genommen die Oberfläche bilden. Diese werden alle innerhalb des Primitives-Abschnitts aufgezählt.

Beispiel: Durch

```

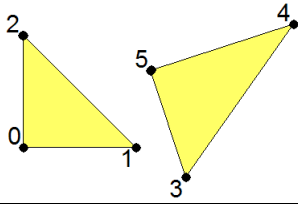
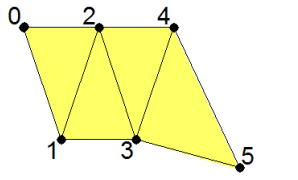
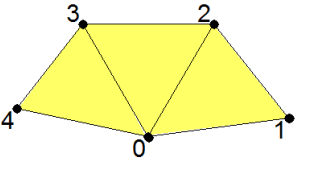
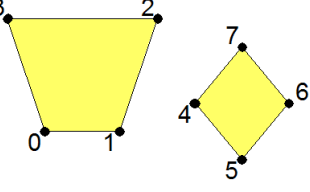
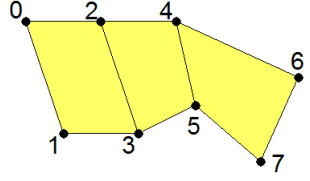
Primitives[0] =
{
    Quads {
        0,1,2,3, 4,6,8,9
    };
    Polygon {
        3,6,7,9,11
    };
};

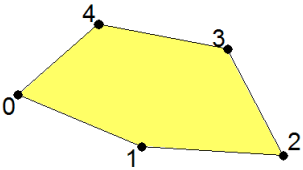
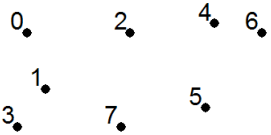
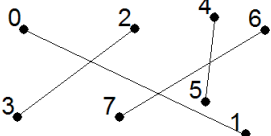
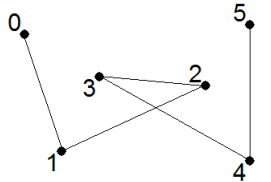
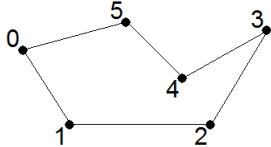
```

wird ein Objekt definiert, das aus zwei Vierecken und einem Polygon (mit fünf Ecken) definiert.

Zusätzlich besteht die Möglichkeit, mehrere Detailstufen des Objekts zu definieren. Dies wird als „Level of Detail“ (LOD) bezeichnet. Dabei ist LOD 0 stets die genaueste Darstellung des Objekts, die höheren LODs 1, 2, ... sind dann immer weiter reduzierte Darstellungen des Objekts. Levels of Detail können definiert werden, indem Abschnitte `Primitives[n]` mit aufsteigenden Werten von `n` definiert werden. Das dargestellte Level of Detail wird abhängig vom Abstand des Objekts zur Kamera und dem angegebenen `LodDistance`-Parameter (siehe Kap. 3.3.3.15) ausgewählt.

Tabelle 1 Primitivtypen

Primitivtyp	Beschreibung	
Triangles	Unzusammenhängende Dreiecke. Je drei Vertices spannen ein einzelnes, texturiertes Dreieck auf.	
TriangleStrip	Zusammenhängender Dreieckszug. Die ersten drei Vertices spannen ein texturiertes Dreieck auf. Jeder weitere Vertex erzeugt ein weiteres, texturiertes Dreieck.	
TriangleFan	Alle Dreiecke teilen sich den ersten Vertex. Die ersten drei Vertices spannen ein texturiertes Dreieck auf und jeder weitere Vertex fügt ein weiteres Dreieckssegment hinzu.	
Primitiv	Beschreibung	
Quads	Je vier Vertices bilden ein einzelnes, texturiertes Viereck. Die Anzahl der Vertices muss demnach immer ein vielfaches von vier betragen.	
QuadStrip	Zusammenhängender Viereckszug. Die ersten vier Vertices spannen ein texturiertes Viereck auf und jedes weitere Vertexpaar fügt ein weiteres Viereck hinzu.	

Primitiv	Beschreibung	
Polygon	Konvexes, ebenes Polygon. SGE ist nicht in der Lage konkave oder unebene Polygone darzustellen. Diese müssen von Hand zerlegt werden.	
Points	Einzelne Punkte. Jeder Vertex wird durch einen einzelnen Punkt dargestellt, der jeweils einen Pixel groß ist.	
Lines	Linienpaare. Immer zwei Vertices bilden eine Linie, die einen Pixel breit ist.	
LineStrip	Eine Serie von zusammenhängenden Linien. Jeder Vektor verlängert die Linie.	
LineLoop	Geschlossener Linienzug. Gleiches Verhalten wie beim LineStrip, nur dass eine zusätzliche Linie zwischen dem letzten und dem ersten Vertex eingefügt wird.	

3.3.8.1 Primitivliste

Alternativ zur Definition von LOD-Primitiven kann auch eine Primitivliste definiert werden. Dabei handelt es sich um ein kompakteres und effizienteres, aber weniger flexibles Format.

Grammatik 14 Primitivlisten

```

<Primitivliste> →
    <Primitivlistentyp> (
        (,) <Zahl>
    );

```

Als Primitivlistentyp kommen in Frage:

Primitivlistentyp	Primitiv	Entsprechender Primitiventyp
-------------------	----------	------------------------------

TRI	Separate Dreiecke	Triangles
STRIP	Dreieckszug	TriangleStrip
FAN	Dreiecksfächer	TriangleFan
QUAD	Separate Vierecke	Quads
QUADSTRIP	Viereckszug	QuadStrip

In der Primitivliste werden dann exakt wie beim entsprechenden Primitiventyp Vertices indiziert und der Reihe nach aufgezählt.

Im folgenden Beispiel werden vier Dreiecke über insgesamt neun Vertices definiert:

```
TRI (
    0, 1, 2,
    2, 3, 0,
    4, 5, 6,
    6, 7, 8
);
```

Alle über Primitivlisten definierten Primitive werden der Detailstufe 0 zugeordnet.

3.3.9 Subobjekte

Grammatik 15 Definition eines Subobjektes

```
<Subobjekt> →
    SubObject <Name> (0...1) <Subobjekt-Erweiterung>
    {
        (0...1) <Texturdefinition>
        (0...*) <Parameter>
        (0...*) <Vertextabelle>
        (0...*) <LOD-Primitive>
        (0...*) <Subobjekt>
    };

<Subobjekt-Erweiterung> →
    : <Objektnamen> |
    : previous |
    : parent
```

In jedes Objekt (und auch rekursiv in seine Subobjekte) können Subobjekte eingebettet werden, die Teile des Objekts darstellen. Für diese können, wie für das Hauptobjekt, eine eigene Textur, Parameter, eine Vertextabelle und die Primitive definiert werden. Üblicherweise wird ein Objekt inhaltlich in übersichtlichere Subobjekte gegliedert. Eine Gliederung muss vorgenommen werden, wenn für einen Teil des Objekts eine andere Textur verwendet werden soll.

Auch Subobjekte können andere Objekte erweitern. Zusätzlich zu der Möglichkeit, ein komplettes anderes SIG-Objekt durch die Angabe eines <Objektnamens> als Subobjekt einzubinden (siehe Abschnitt 3.3.1), bestehen für Subobjekte zwei zusätzliche Möglichkeiten:

- Durch die Angabe von **parent** als Erweiterungsquelle stellt das Subobjekt eine Kopie seines Vaterobjekts dar. In diesem Fall sollte das Subobjekt in der SIG-Datei nach der Angabe aller Parameter und der Definition der Geometrie des Vaterobjekts stehen. Die Transformation (**Translate**, **Rotate**, **Scale**, siehe

Kap. 3.3.3.10) des Vaterobjekts wird in diesem Fall ausnahmsweise nicht übernommen, da die meisten Animationen rekursiv arbeiten und die Transformation des Vaterobjekts daher ohnehin schon auf das neu definierte Subobjekt angewandt wird.

- Durch die Angabe von **previous** als Erweiterungsquelle stellt das Subobjekt eine Kopie des in der SIG-Datei direkt vor ihm stehenden Subobjekts dar. Dies ist nur zulässig, falls das **previous**-Subobjekt dasselbe Vaterobjekt hat wie das neu definierte Subobjekt.

Auch in diesen Fällen gilt, dass Objekte, die andere erweitern, keine neue Geometrie, Primitive oder Subobjekte definieren dürfen.

3.4 Beispielobjekt

3.4.1 Einfaches Objekt: Haus

Das folgende Skript zeigt die Definition eines einfachen Haus im SGE-Format:

```

define SGE3DObject House
{
    Type = (3,2,0);
    AlphaTest = 0.2;
    Color = (1,1,1,1);
    Culling = CCW;
    DepthTest = 1;
    Fog = 1;
    RenderClass = ZBuffered;
    Show = true;
    Texture = house.house1;
    TextureFilter = ClampS | ClampT | MagLinear | MinLinearMipMapLinear | TextureMipMap;
    Vertices =
    {
        ([0.01,0.79],<-5.5,-8.5,6>),
        ([0.01,-0.65],<-5.5,-8.5,-5>),
        ([0.99,-0.65],<-5.5,8.5,-5>),
        ([0.99,0.79],<-5.5,8.5,6>),
        ([0.01,0.79],<5.5,8.5,6>),
        ([0.01,-0.65],<5.5,8.5,-5>),
        ([0.99,-0.65],<5.5,-8.5,-5>),
        ([0.99,0.79],<5.5,-8.5,6>),
        ([0.25,1],<0,-3.3,9>),
        ([0.01,0.79],<-5.5,-8.5,6>),
        ([0.99,0.79],<-5.5,8.5,6>),
        ([0.75,1],<0,3.3,9>),
        ([0.25,1],<0,3.3,9>),
        ([0.01,0.79],<5.5,8.5,6>),
        ([0.99,0.79],<5.5,-8.5,6>),
        ([0.75,1],<0,-3.3,9>),
        ([0.01,0.79],<-5.5,8.5,6>),
        ([0.01,-0.65],<-5.5,8.5,-5>),
        ([0.39,-0.65],<5.5,8.5,-5>),
        ([0.39,0.79],<5.5,8.5,6>),
        ([0.01,0.79],<5.5,-8.5,6>),
        ([0.01,-0.65],<5.5,-8.5,-5>),
        ([0.39,-0.65],<-5.5,-8.5,-5>),
        ([0.39,0.79],<-5.5,-8.5,6>),
        ([0.76,0.79],<-5.5,-8.5,6>),
        ([0.875,1],<0,-3.3,9>),
        ([0.99,0.79],<5.5,-8.5,6>),
        ([0.76,0.79],<5.5,8.5,6>),
        ([0.875,1],<0,3.3,9>),
        ([0.99,0.79],<-5.5,8.5,6>)
    };
    Primitives[0] =
    {
        Quads
        {
            0,1,2,3,
            4,5,6,7,
            8,9,10,11,
            12,13,14,15,
            16,17,18,19,
            20,21,22,23
        };
        Triangles
        {
            24,25,26,27,
            28,29
        };
    };
};
};

```

Es wird folgende Textur verwendet:



Abbildung 9: Textur house.house1

4 SGEOBJECTPREV

4.1 Überblick

Die Anwendung **SGEObjectPrev** dient dazu, mit SGE-Objektdateien und Textur-Bildern entworfene SGE-Objekte zu betrachten. Die Benutzeroberfläche ist in Abbildung 10 dargestellt.

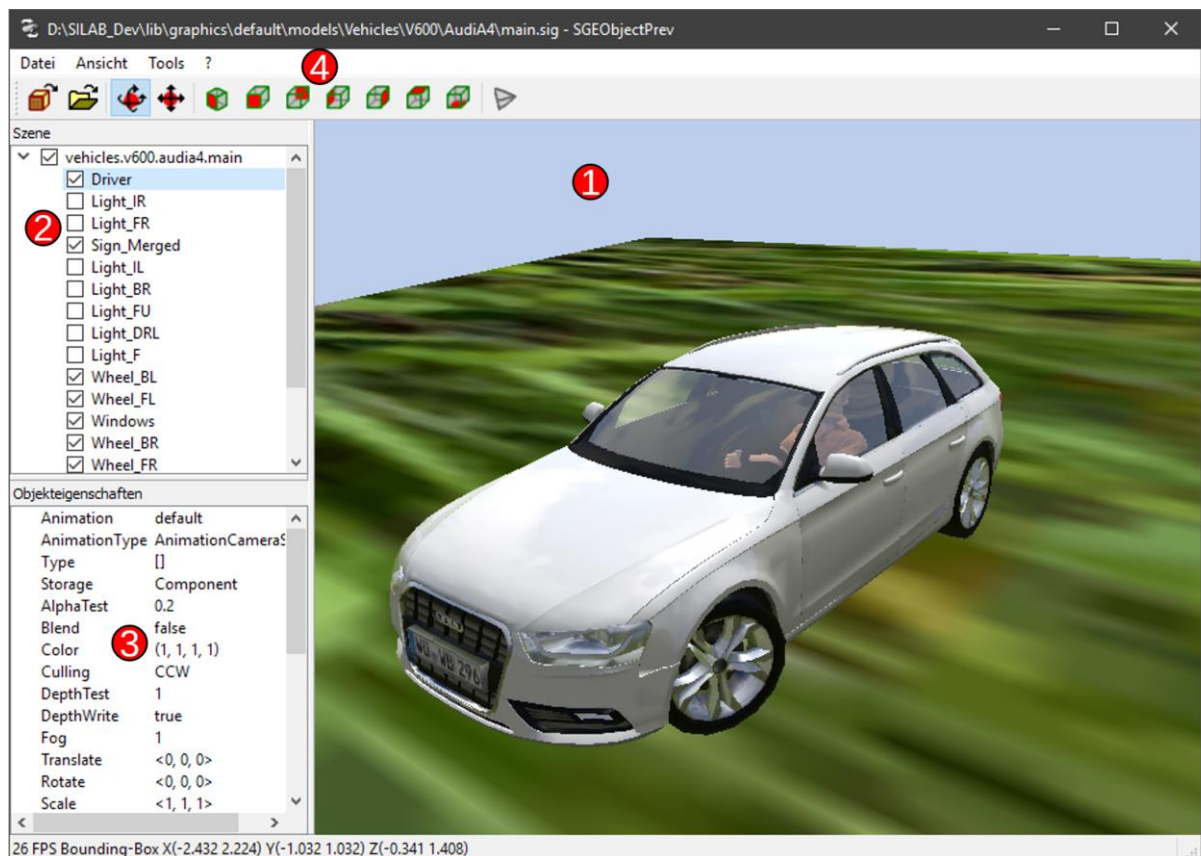


Abbildung 10: SGEObjectPrev mit einem geladenen SGE-Objekt (AudiA4) und einem Hilfsobjekt (Wiese/Flach)

Wichtige Elemente der Benutzeroberfläche werden nun genauer beschrieben.

1 Vorschaufenster

Im Vorschaufenster wird das geladene SGE-Objekt (und ggf. die zusätzlich geladenen Hilfsobjekte) dargestellt.

2 Szene

In der Szenenansicht wird der Name aller geladenen SGE-Objekte dargestellt. Zusätzlich wird die Struktur der Objekte, d.h. ihre Einteilung in Unterobjekte, dargestellt. (Nähere Informationen zu Unterobjekten erhalten Sie in Kap. 3.2.1.) Über die Kon-

trollkästchen kann jedes Objekt bzw. Unterobjekt einzeln ein- und ausgeblendet werden. Zusätzlich können durch Doppelklick die Namen der Unterobjekte geändert werden.

Hinweis: Um die Namensänderung dauerhaft zu übernehmen, muss das Objekt anschließend über den Menübefehl *Datei* → *Speichern unter...* gespeichert werden.

3 Objekteigenschaften

In dieser Ansicht werden die Werte aller Parameter des Objekts angezeigt, das in der Szenenansicht ausgewählt ist. Durch einen Doppelklick können die meisten Parameter auch geändert werden.

Hinweis: Aus technischen Gründen können *VertexShader* und *PixelShader* nicht geändert werden. Änderungen einiger anderer Parameter, z.B. der Animation, haben auf die Darstellung in *SGEObjectPrev* keinen Einfluss.

Hinweis: Um die Änderungen dauerhaft zu übernehmen, muss das Objekt anschließend über den Menübefehl *Datei* → *Speichern unter...* gespeichert werden.

4 Werkzeugleiste

In der Werkzeugleiste finden Sie Befehle zum Laden von Objekten sowie zur Navigation durch die Szenerie. Diese sind in Kap. 4.4 näher beschrieben.

4.2 Laden eines Objekts

Über den Menübefehl *Datei/Öffnen* kann eine Objektdefinitionsdatei (*.sig oder *.sge) in dem in Abschnitt 3.3 beschriebenen Format geladen werden. Das Objekt wird unmittelbar danach im Fenster der Anwendung dargestellt.

Wird die Objektdatei während der Darstellung in **SGEObjectPrev** (mithilfe eines externen Text-Editors) verändert, dann kann die geänderte Version mit einem Doppelklick in das Darstellungsfenster erneut geladen werden.

Das Objekt kann über den Schalter  im Wireframe-Modus betrachtet werden.

Über den Menübefehl *Datei/Hilfsobjekt laden* kann ein zusätzliches SGE-Objekt geladen werden, das im Hintergrund dargestellt wird.

4.2.1 „Nach dem Laden des Objekts sehe ich nichts!“

Gelegentlich scheint sich durch das Laden eines Objekts im *SGEObjectPrev* nichts zu ändern – Sie sehen weiterhin nur einen blauen Hintergrund.

Bitte prüfen Sie in diesem Fall folgendes:

- Wurden beim Laden des Objekts Fehler im Logfenster notiert? (siehe Kap. 4.3)
- Möglicherweise ist das Objekt größer als der standardmäßig sichtbare Ausschnitt. Entfernen Sie die Kamera vom Ursprung, indem Sie mit der rechten Maustaste ins Grafikfenster klicken und dann die Maus nach unten ziehen (Siehe Kap. 4.4, „Sphärische Navigation“).

4.3 Warn- und Fehlermeldungen

Falls das Objekt fehlerhaft definiert ist oder einige verwendete Ressourcen (z.B. Texturbilder) nicht gefunden werden, werden Fehlermeldungen im Logfenster ausgege-

ben, das unter *Ansicht/Logfenster* aufgerufen werden kann. Wie im unteren Teil des SILAB-Hauptfensters werden Warnungen (z.B. nicht gefundene Textur) mit  markiert, Fehler (z.B. Syntaxfehler im SGE-Objekt) mit .

4.4 Navigation

Das Objekt kann in zwei verschiedenen Modi mit der Maus bewegt werden. Die Modi werden über die entsprechenden Schalter in der Toolbar von SGEObjectPrev aktiviert. Befindet man sich in einem Modus, so kann bei gedrückter Strg-Taste temporär der jeweils andere Modus aktiviert werden.



Sphärische Navigation:

Bei gedrückter linker Maustaste und Bewegung der Maus nach links/rechts rotiert die Kamera in der horizontalen Ebene um ein Rotationszentrum. Das Rotationszentrum kann mit der translatorischen Navigation verändert werden. Beim Start von SGEObjectPrev entspricht das Rotationszentrum dem Ursprung des SGE-Koordinatensystems.

Bei gedrückter linker Maustaste und Bewegung der Maus nach oben/unten rotiert die Kamera in der vertikalen Ebene um das Objekt. Auch hier ändert sich das Rotationszentrum nicht.

Bei gedrückter rechter Maustaste und Bewegung der Maus nach oben/unten wird der Abstand der Kamera zum Rotationszentrum verändert. Den selben Effekt hat die Betätigung des Mousrads.



Translatorische Navigation:

Bei gedrückter linker Maustaste und Bewegung der Maus nach links/rechts bzw. oben/unten wird die Kamera nach links/rechts bzw. oben/unten verschoben.

Bei gedrückter rechter Maustaste und Bewegung der Maus nach oben/unten wird die Kamera in das Bild hinein/aus dem Bild heraus verschoben. Denselben Effekt hat die Betätigung des Mousrads.



Automatische Kamerapositionierung:

Die dargestellte Schaltfläche positioniert die Kamera so, dass das komplette Objekt sichtbar ist.

Hinweis: Bei sehr großen oder ungewöhnlich geformten Objekten kann die automatische Kamerapositionierung fehlschlagen.



Spezielle Kamerapositionen:

Die dargestellten Schaltflächen positionieren die Kamera an speziellen Stellen. Das Objekt kann von vorne, von hinten, von der linken Seite, von der rechten Seite, von oben und von unten betrachtet werden. Der aktuelle Abstand der Kamera vom Objekt wird dabei nicht verändert. Die Kamera blickt dabei immer auf den Ursprung des Koordinatensystems.

Speichern/Wiederherstellen von Kamerapositionen:

Die aktuelle Kameraposition kann gespeichert und wiederhergestellt werden. Für diese Bookmarks gibt es 4 Speicherplätze. Das Speichern erfolgt über die Menübe-

fehle *Ansicht/Kameraposition speichern/1* bis *Ansicht/Kameraposition speichern/4* oder die Tastenkombinationen *Strg+F5* bis *Strg+F8*. Das Wiederherstellen erfolgt über die Menübefehle *Ansicht/Kameraposition laden/1* bis *Ansicht/Kameraposition laden/4* oder die Tasten *F5* bis *F8*.

4.5 Import von 3D-Modellen

SGEObjectPrev unterstützt den Import von 3D-Modellen in diversen 3D-Austauschformaten. Im Folgenden werden die unterstützten Formate aufgezählt. Die am besten unterstützten Formate stehen am Anfang der Liste. Die meisten 3D-Modeller unterstützen das Speichern in mehreren dieser Formate, versuchen Sie diese also in dieser Reihenfolge.

- FBX (Autodesk Filmbox)
- OBJ (Wavefront Object)
- 3DS (Autodesk 3DS)
- DAE (Collada File Format)

Bitte beachten Sie allerdings, dass SGEObjectPrev nur polygonale Geometrie importieren kann. Der Import von parametrischen Flächen (z.B. NURBS) wird nicht unterstützt. Damit die Modelle die Simulation nicht überfordern, sollte die Geometrie nicht allzu kompliziert sein, als grobe Richtlinie sollten Objekte aus weniger als 20000 Dreiecken bestehen.

Die Vorgehensweise ist dann wie folgt:

- Speichern Sie das Modell mit all seinen Texturen in einem Verzeichnis, z.B. D:\Models\Building.
- Konvertieren Sie ggf. alle Texturen in ein Format, das von SILAB verwendet werden kann (vorzugsweise PNG, alternativ auch TGA oder JPG) und ändern Sie das Modell in Ihrem 3D-Modeller so ab, dass die konvertierten Texturen verwendet werden⁸.
- Stellen Sie auch sicher, dass die Höhe und Breite aller Texturen eine Zweierpotenz ist⁹, und skalieren Sie die Bilder ggf. mit einem Bildbearbeitungsprogramm entsprechend.
- Im Verzeichnis sollten nun folgende Dateien vorhanden sein:
 - Die Objektdatei, z.B. MeinObjekt.obj
 - Im Fall von Modellen im OBJ-Format: Die Materialdefinitionsdatei, z.B. MeinObjekt.mtl
 - Die Texturdateien, z.B. stein.png
- Starten Sie nun SGEObjectPrev.
- Normalerweise sollten Sie das Objekt im Verzeichnis des Projekts abspeichern, für das es verwendet werden soll. Rufen Sie daher zunächst *Datei/Projekt auswählen...* auf und wählen Sie den Projektordner aus (z.B. SILAB\Projects\MeinProjekt).

⁸ Falls das Modell als OBJ-Datei vorliegt, können Sie den Pfad und Dateinamen auch mithilfe eines Texteditors anpassen. Öffnen Sie dazu die Datei „MeinObjekt.mtl“ und passen Sie die mit „map_“ beginnenden Zeilen entsprechend an.

⁹ 1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024 oder 2048 Pixel.

- Starten Sie nun den Import mit *Datei/3D-Objekt importieren...* und wählen Sie die Objektdatei (z.B. *D:\Models\Building\MeinObjekt.obj*) aus.
- Führen Sie weitere Einstellungen im nachfolgend beschriebenen Dialogfeld durch.

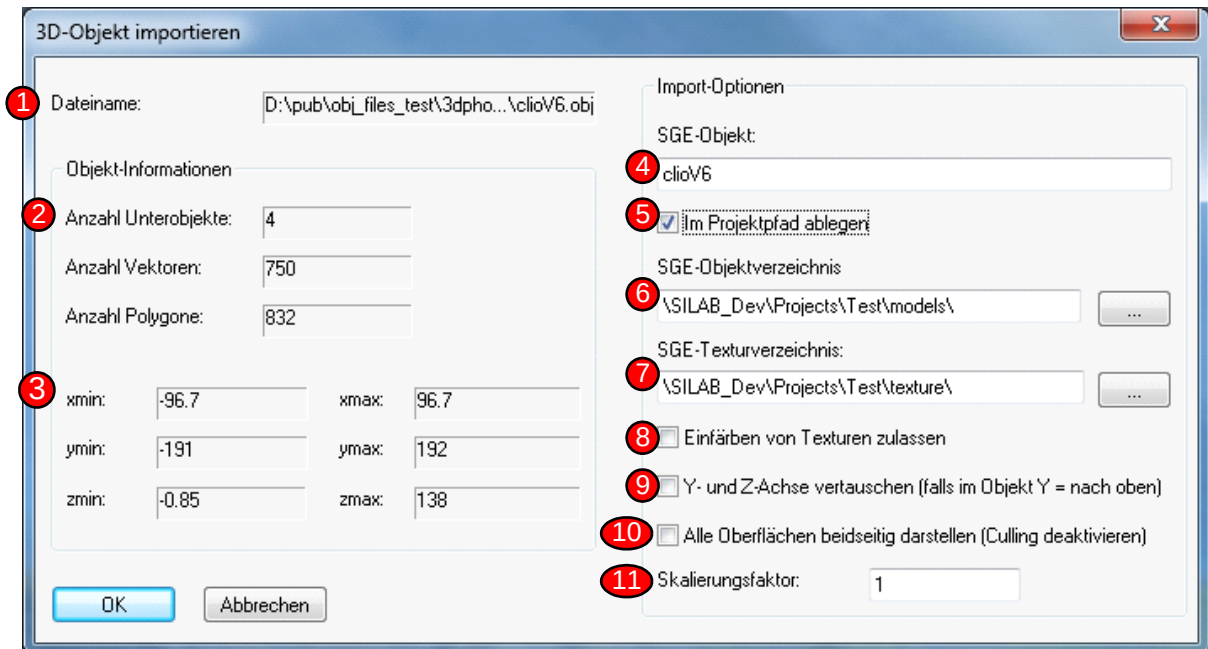


Abbildung 11 Import von 3D-Modellen

1 Dateiname

Hier wird der Dateiname des importierten 3D-Modells angezeigt.

2 Objekt-Informationen

Enthält Informationen über die Struktur des Objekts und seine Geometrie.

3 Bounding Box

Zeigt die Dimensionen des Objekts (im Koordinatensystem des importierten 3D-Modells) an.

4 SGE-Objektname

Hier kann ein Name für die erzeugte SGE-Objektdatei angegeben werden. Die Dateierweiterung „.sge“ soll dabei **nicht** angegeben werden.

5 Projektspezifischer Import

Wenn aktiviert, wird das Objekt im aktuell ausgewählten Projektverzeichnis abgespeichert. (Dies wird für benutzerdefinierte Objekte empfohlen.)

Wenn deaktiviert, wird das Objekt im Standard-SILAB-Modellverzeichnis abgelegt.

6 SGE-Objektverzeichnis

Hier kann ausgewählt werden, in welchem Unterverzeichnis die SGE-Objektdatei abgelegt wird. Falls das Verzeichnis noch nicht existiert, wird es beim Import erstellt.

7 SGE-Texturverzeichnis

Hier kann ausgewählt werden, in welchem Unterverzeichnis die Texturdateien nach dem Import abgelegt werden. Falls das Verzeichnis noch nicht existiert, wird es beim Import erstellt.

8 Einfärben von Texturen zulassen

Wenn aktiviert, wird auch bei Materialien, die eine Textur verwenden, der Farbwert verwendet. Dies führt allerdings häufig dazu, dass das Material zu dunkel dargestellt wird.

Wenn deaktiviert, wird bei texturierten Materialien die Objektfarbe auf „weiß“ gesetzt.

9 Y- und Z-Achse vertauschen

Wenn aktiviert, wird beim Import die Y- und Z-Achse vertauscht. Verwenden Sie diese Einstellung, wenn im Modeller Y nach oben zeigt (dies ist häufig der Fall) bzw. wenn Sie anhand der Bounding Box **3** feststellen können, dass Y die vertikale Achse ist. (In SILAB zeigt die Z-Achse nach oben.)

10 Alle Oberflächen beidseitig darstellen

Wenn aktiviert, wird kein „Culling“ der Oberflächen verwendet. Verwenden Sie diese Einstellung nur dann, wenn Objekte fälschlicherweise nur „von innen heraus“ sichtbar sind bzw. Sie nur die Rückseite des Objekts sehen.

11 Skalierungsfaktor

Geben Sie hier ein, mit welchem Faktor die Vertex-Koordinaten des 3D-Modells multipliziert werden sollen.

Bitte beachten Sie hierbei die Dimensionen des Objekts **3** und berücksichtigen Sie, dass in **SGE-Objekten** die **Einheit Meter** verwendet wird.

Häufig verwendete Skalierungsfaktoren sind:

0.0254 wenn das Modell in der Einheit Zoll vorliegt

0.01 wenn das Modell in der Einheit cm vorliegt

0.001 wenn das Modell in der Einheit mm vorliegt

12 Objekt beim Import optimieren

Wenn aktiviert, wird die Geometrie des Objekts beim Import importiert, so dass es anschließend effizienter von der Grafikkarte dargestellt werden kann. Die Dauer des Importvorgangs kann sich allerdings verlängern.

Nach dem Klick auf „OK“ wird das Objekt importiert und im SGEObjectPrev geladen sowie automatisch unter dem angegebenen Dateinamen abgespeichert.

4.5.1 „Nach dem Import des Objekts sehe ich nichts!“

Gelegentlich scheint sich durch den Import eines Objekts im SGEObjectPrev nichts zu ändern – Sie sehen weiterhin nur einen blauen Hintergrund.

Bitte prüfen Sie in diesem Fall folgendes:

- Wurden beim Import des Objekts Fehler im Logfenster notiert? (siehe Kap. 4.3)
- Möglicherweise ist das Objekt größer als der standardmäßig sichtbare Ausschnitt. Entfernen Sie die Kamera vom Ursprung, indem Sie mit der rechten Maustaste ins Grafikfenster klicken und dann die Maus nach unten ziehen (Siehe Kap. 4.4, „Sphärische Navigation“).

4.5.2 Spezielle Funktionen in 3D-Modellen

Im FBX-Dateiformat können einige erweiterte Funktionen verwendet werden, um den Import der 3D-Modelle in SILAB zu verbessern.

- Es können *User Properties* für die Teilobjekte des 3D-Modells gesetzt werden. Falls eine User Property **Name** = <Zahl> gesetzt ist, wird dadurch beim Import der SIG-Parameter des gleichen Namens festgelegt (siehe Kap. 3.3.3). Dies ermöglicht die Einstellung von SIG-Parametern direkt im 3D-Modeller, die nicht vom FBX-Dateiformat unterstützt werden, wie z.B. den *AlphaTest*-Parameter (siehe Kap. 3.3.3.4). Falls es keinen SIG-Parameter mit dem angegebenen Namen gibt, wird stattdessen ein benutzerdefinierter Parameter (siehe Kap. 3.3.6) angelegt. Falls eine User Property **Shader.Name** = <Zahl> gesetzt ist, wird dadurch der Shader-Parameter *Name* festgelegt (siehe Kap. 3.3.5).

4.6 Tools zur Nachbearbeitung

Nachdem ein Objekt importiert wurde, muss es meist noch nachbearbeitet werden. Für viele häufige Nachbearbeitungsaufgaben gibt es in *SGEObjectPrev* bereits Werkzeuge. Im Folgenden werden einige davon beschrieben.

Hinweis: Um die Änderungen dauerhaft zu übernehmen, muss das Objekt anschließend über den Menübefehl *Datei* → *Speichern unter...* gespeichert werden.

4.6.1 Objekt transformieren...

Das Werkzeug „Objekt transformieren“ bewegt, rotiert und / oder verschiebt eines oder mehrere Teilobjekte des geladenen Objekts. Es ist beispielsweise hilfreich, um Objekte an der gewünschten Position darzustellen, sowie um ihre Größe zu korrigieren (evtl. im Vergleich zu einem gleichzeitig geladenen Hilfsobjekt).

Bei der Auswahl des Werkzeugs erscheint ein Dialog, indem Sie zunächst die zu bearbeitenden Teilobjekte auswählen. Sie können das Teilobjekt am leichtesten identifizieren, indem Sie in der Szenenansicht probeweise einzelne Objekte aus- und einblenden. Geben Sie dann Werte für die Translation (Verschiebung), Rotation sowie Skalierung an.

Hinweis: Das verwendete Koordinatensystem ist in Kap. 3.2.3 beschrieben.

4.6.2 Objekt XY-zentrieren

Das Werkzeug „Objekt XY-zentrieren“ zentriert das komplette Objekt mit einem Klick in der XY-Ebene¹⁰. Wenn das Objekt in ein Szenario eingefügt wird, ist somit sichergestellt, dass sein Schwerpunkt auch genau an der vorgesehenen Stelle landet.

4.6.3 Subobjekte verschmelzen...

Mit dem Werkzeug „Subobjekte verschmelzen“ können Sie mehrere Subobjekte, die dieselbe Textur verwenden, miteinander verschmelzen. (Dies verbessert später die Darstellungsleistung, wenn viele solcher Objekte in der Simulation verwendet werden.) Wählen Sie dazu zunächst eines der Subobjekte aus, in das nun die anderen hineinverschmolzen werden sollen. Geben Sie schließlich einen neuen Dateinamen für das überarbeitete Objekt an.

¹⁰ Die XY-Ebene ist die Ebene, die den Boden bildet. Das Objekt wird also durch das XY-Zentrieren nur horizontal verschoben, aber niemals vertikal.

4.6.4 CFG kopieren

Dieser Befehl kopiert Daten in die Zwischenablage, die benötigt werden, um das aktuell geladene Objekt in SILABAEEdit verwenden zu können.

Fügen Sie die Daten anschließend in einem Texteditor in eine .cfg-Datei ein und laden Sie diese in SILABAEEdit, wie im Handbuch SILABAEEdit (Kapitel „Erweiterung um einzelne grafische Objekte“) beschrieben.

4.6.5 Batch-Import für AEdit

Dieser Befehl erzeugt Vorschaubilder und eine CFG-Datei für SILABAEEdit, die alle grafischen Objekte umfasst, die in einem bestimmten Verzeichnis gespeichert sind.

5 GENERIERUNG VON SCREENSHOTS

Die DPUSGEView bietet eine komfortable Möglichkeit zur Erstellung von Screenshots während der Simulation. Diese werden im Format PNG in einem frei wählbaren Verzeichnis abgespeichert. Dazu muss die Versuchs-Konfiguration¹¹ wie folgt erweitert werden:

```
VISFRONT.sgeview.ScreenshotDir = "\\SILAB\\Data\\Screenshots";  
VISFRONT.sgeview.ScreenshotBasename = "Shot";
```

Zusätzlich sollte an den Eingang `VISFRONT.sgeview.Screenshot` ein Switch mit Typ "Taster" angeschlossen werden. Bei jedem Druck auf diesen Taster wird dann ein Screenshot erzeugt und mit nummerierten, fortlaufenden Dateinamen abgelegt. Die Dateien heißen `\\SILAB\\Data\\Screenshots\\Shot0001.png` usw.

Die erzeugten Screenshots werden im Hintergrund – während die Simulation weiterläuft – abgespeichert. Wird der Switch `Screenshot` sehr häufig, d.h. mehr als 1x pro Sekunde, ausgelöst, dann führt dies zu hohem Rechenaufwand und die laufende Simulation kann beeinträchtigt werden.

¹¹ ausgehend von der Beispiel-Konfiguration aus Kap. 2.1.2

6 GENERIERUNG DIGITALER VIDEOS

Zur Generierung digitaler Videos einer Simulationsfahrt kann die DPUSGECapture verwendet werden. Beispiele zur Anwendung dieser DPU finden Sie im separaten „Tutorial Video-Aufzeichnung“.